



人工智能原理与算法

第5章-对抗博弈

中国科学院自动化研究所
国科大人工智能学院

雷震

- 博弈
- 博弈中的优化决策
- α - β 剪枝
- 不完美的实时决策
- 随机博弈
- 部分可观察的博弈
- 博弈搜索算法的局限性

5.1 博弈

■ 双人完备信息博弈：

- 指两位选手对垒，轮流走步，这时每一方不仅都知道对方过去已经走过的棋步，而且还能估计出对方未来可能的走步；
- 对弈的结果是一方赢（另一方则输），或者双方和局；
- 这类博弈的实例有：一字棋、余一棋、西洋跳棋、国际象棋、中国象棋、围棋等。

■ 机遇性博弈：存在不可预测性的博弈，例如掷币、西洋双陆棋等。

5.1 博弈

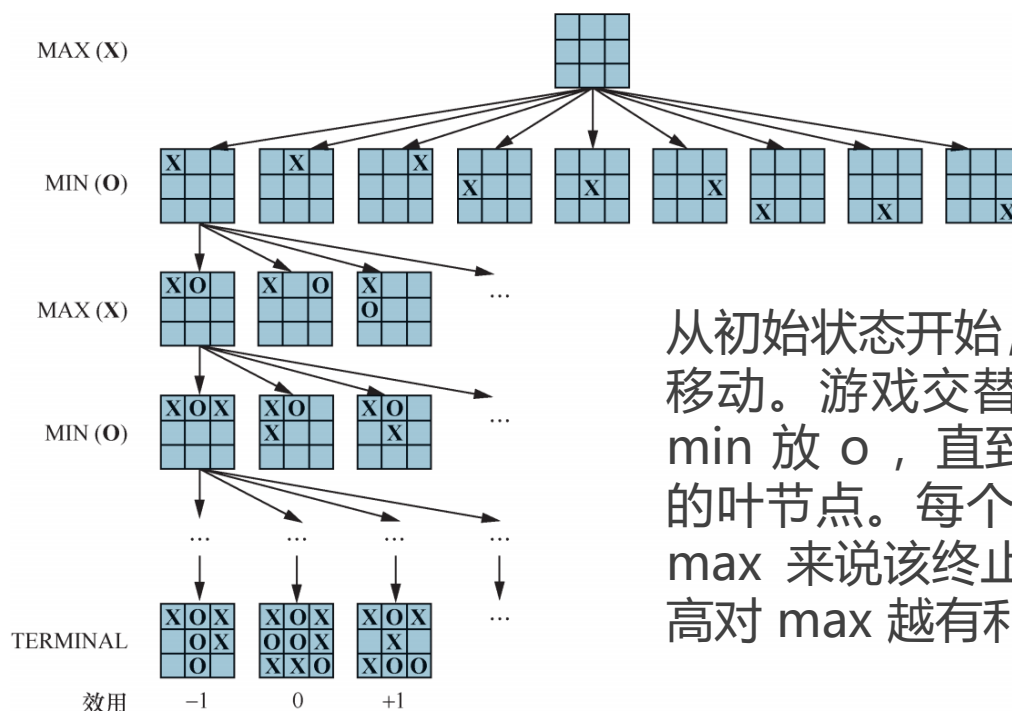
■ 博弈形式化

- 将两个参与者分别称为 $\max$ 和 $\min$ ，使用以下元素对博弈进行形式化定义。
 - **S_0** ：初始状态，指定博弈开始时如何设置。
 - **To-Move(s)**：在状态 s 下，轮到其移动的参与者。
 - **Actions(s)**：在状态 s 下，全体合法移动的集合。
 - **Result(s, a)**：[转移模型](#)，定义状态 s 下执行动作 a 所产生的结果状态。
 - **Is-Terminal(s)**：[终止测试](#) (terminal test)，博弈结束时返回真，否则返回假。博弈结束时的状态称为[终止状态](#) (terminal state)。
 - **Utility(s, p)**：效用函数（也称为目标函数或收益函数），定义博弈结束时终止状态 s 下参与者 p 得到的最终的数值收益。

5.1 博弈

■ 博弈形式化

- 初始状态、Actions 函数和 Result 函数定义了状态空间图——其中顶点表示状态，边表示移动。



从初始状态开始，max 有 9 种可能的移动。游戏交替进行，max 放 x，min 放 o，直到到达对应于终止状态的叶节点。每个叶节点上的数字是对 max 来说该终止状态的效用值，值越高对 max 越有利，对 min 越不利。

图 5-1 井字棋的（部分）博弈树。最上面的节点是初始状态，MAX 先移动，在某个空位上放一个 x。我们展示了树的一部分，给出 MIN(O) 和 MAX(X) 的交替移动，直到最终到达终止状态，根据博弈规则为终止状态分配效用值

- 博弈
- 博弈中的优化决策
- α - β 剪枝
- 不完美的实时决策
- 随机博弈
- 部分可观察的博弈
- 博弈搜索算法的局限性

5.2 博弈中的优化决策

- 一般搜索问题中，最优解是到达目标状态的一系列行动，终止状态即为取胜。
- 对抗搜索中，MIN在博弈中也有发言权
 - MAX必须找到应急策略，制定出MAX初始状态下应该采取的行动
 - 接着是MIN行棋
 - MAX再行棋要考虑到MIN的每种可能的回应
 - 依次类推
- 当对手不犯错误时，最优策略能够得到至少不比任何其他策略差的结果。
- 对于具有多个结果分数的博弈，需要一种更一般的算法，即**极小化极大搜索**。

5.2 博弈中的优化决策

- 给定博弈树，可以通过计算树中每个状态的极小化极大值确定最优策略，记为Minimax(s)。

MINIMAX(s) =

$$\begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{如果 Is-TERMINAL}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{如果 To-MOVE}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{如果 To-MOVE}(s) = \text{MIN} \end{cases}$$

可能有一些策略在对付非最优化对手方面做的比极小极大策略好，但是用这些策略对付最优化对手则会得到更差的结果。

5.2 博弈中的优化决策

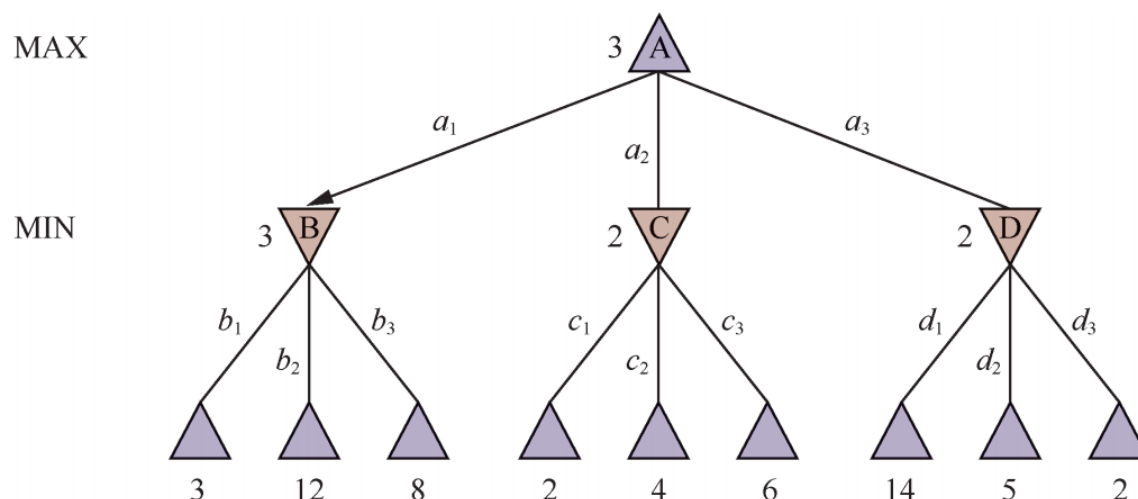
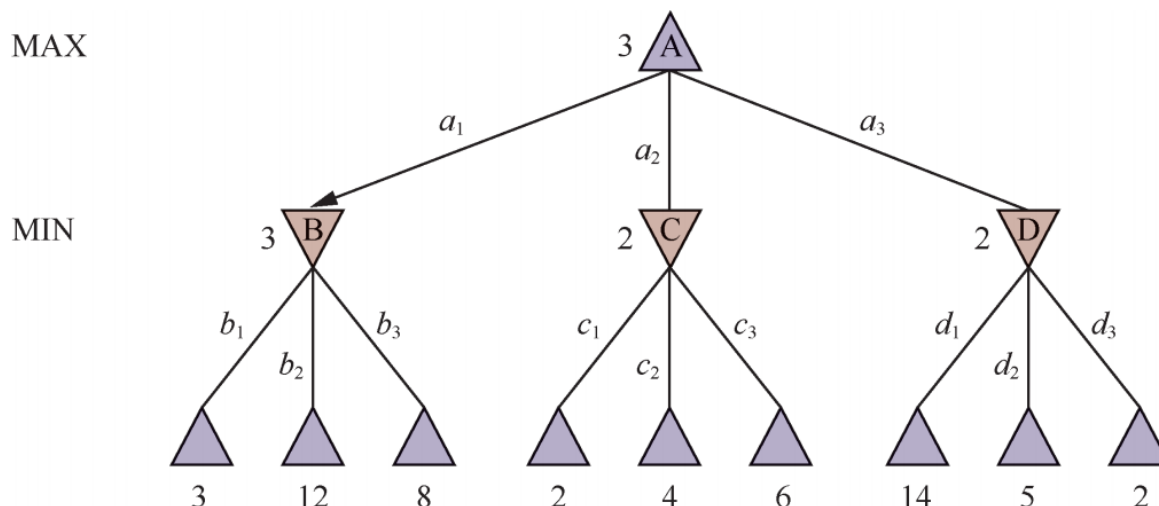


图 5-2 二层博弈树。 $\triangle$ 节点为“MAX 节点”，即轮到 MAX 移动， ∇ 节点为“MIN 节点”。终止节点显示 MAX 的效用值，其他节点标记有它们的极小化极大值。MAX 在根节点的最佳移动是 a_1 ，因为它指向极小化极大值最高的状态，而 MIN 的最佳响应是 b_1 ，因为它指向极小化极大值最低的状态

AND-OR搜索算法:MAX类似于OR结点, MIN类似于AND结点

5.2 博弈中的优化决策



□ 第一个结点标记为B

- 三个后继的值分别是3、12、8，极小极大值是3。
- 其他两个MIN的极小极大值都是2

□ 根是MAX结点

- 后继结点的极小极大值分别为3、2、2，极小极大值是3。

□ 根结点的极小极大对策

- a_1 是最优选择，因为它指向最高的极小极大值的终止状态。

5.2 博弈中的优化决策

■ 极小化极大算法:

- 极小化极大算法从当前状态计算极小极大决策。
- 将Minimax(s)转化为一个搜索算法，即尝试所有动作然后选择其结果状态的 Minimax 值最大的动作作为 max 的最佳移动。
- 这是一种递归算法，它一直向下进行到叶节点，然后随着递归的展开通过搜索树**倒推**极小化极大值。

5.2 博弈中的优化决策

```
function MINIMAX-SEARCH(game, state) returns 一个动作
  player  $\leftarrow$  game.To-Move(state)
  value, move  $\leftarrow$  MAX-VALUE(game, state)
  return move
```

5.2 博弈中的优化决策

```
function MAX-VALUE(game, state) returns 一个(utility, move)对
  if game.Is-TERMINAL(state) then return game.UTILITY(state, player), null
  v, move  $\leftarrow -\infty$ , null
  for each a in game.ACTIONS(state) do
    v2, a2  $\leftarrow$  MIN-VALUE(game, game.RESULT(state, a))
    if v2 > v then
      v, move  $\leftarrow$  v2, a
  return v, move
```


5.2 博弈中的优化决策

```
function MIN-VALUE(game, state) returns 一个(utility, move)对
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v, move  $\leftarrow +\infty$ , null
  for each a in game.ACTIONS(state) do
    v2, a2  $\leftarrow$  MAX-VALUE(game, game.RESULT(state, a))
    if v2 < v then
      v, move  $\leftarrow$  v2, a
  return v, move
```

5.2 博弈中的优化决策

```
function MINIMAX-SEARCH(game, state) returns 一个动作
    player  $\leftarrow$  game.TO-MOVE(state)
    value, move  $\leftarrow$  MAX-VALUE(game, state)
    return move
```

```
function MAX-VALUE(game, state) returns 一个(utility, move)对
    if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
    v, move  $\leftarrow -\infty$ , null
    for each a in game.ACTIONS(state) do
        v2, a2  $\leftarrow$  MIN-VALUE(game, game.RESULT(state, a))
        if v2 > v then
            v, move  $\leftarrow$  v2, a
    return v, move
```

```
function MIN-VALUE(game, state) returns 一个(utility, move)对
    if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
    v, move  $\leftarrow +\infty$ , null
    for each a in game.ACTIONS(state) do
        v2, a2  $\leftarrow$  MAX-VALUE(game, game.RESULT(state, a))
        if v2 < v then
            v, move  $\leftarrow$  v2, a
    return v, move
```

图 5-3 使用极小化极大计算最优移动的算法。最优移动是指，在假定对手移动是为了使效用值最小的前提下，使终止状态效用值最大的移动。函数 MAX-VALUE 和 MIN-VALUE 遍历整个博弈树直到叶节点，以确定每个状态的倒推值以及如何移动以到达该状态

5.2 博弈中的优化决策

■ 极小化极大算法:

- 执行完整的深度优先探索。假设树的最大深度是 m ，并且在每个点都有 b 种合法移动，那么极小化极大算法的时间复杂度为 $O(b^m)$ 。
- 一次性生成所有动作的算法，空间复杂度为 $O(bm)$ 。
- 每次生成一个动作的算法，空间复杂度是 $O(m)$ 。

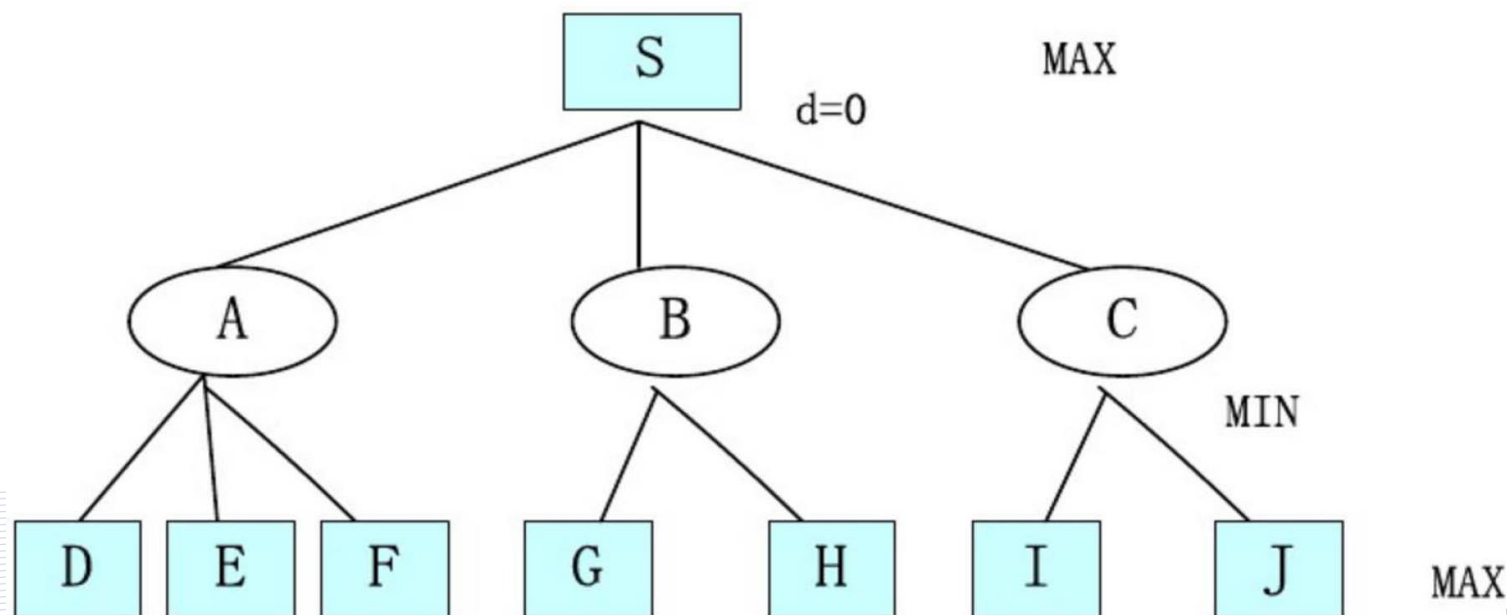
不实用，但可作为对博弈进行数学分析和设计实用算法的基础

5.2 博弈中的优化决策

- **对于人类下棋的方法：**实际上采用的是一种试探性方法
 - ✓ 首先假定走了一步棋，看对方会有哪些应法，然后再根据对方的每一种应发，看我方是否有好的回应
 - ✓ 这一过程一直进行下去，直到若干步以后，找到了一个满意的走法为止
 - ✓ 初学者可能只能看一、两个回合，而高手则可以看几个，甚至十几个回合
- **极小化极大搜索方法：**模拟的就是人的这样一种思维过程

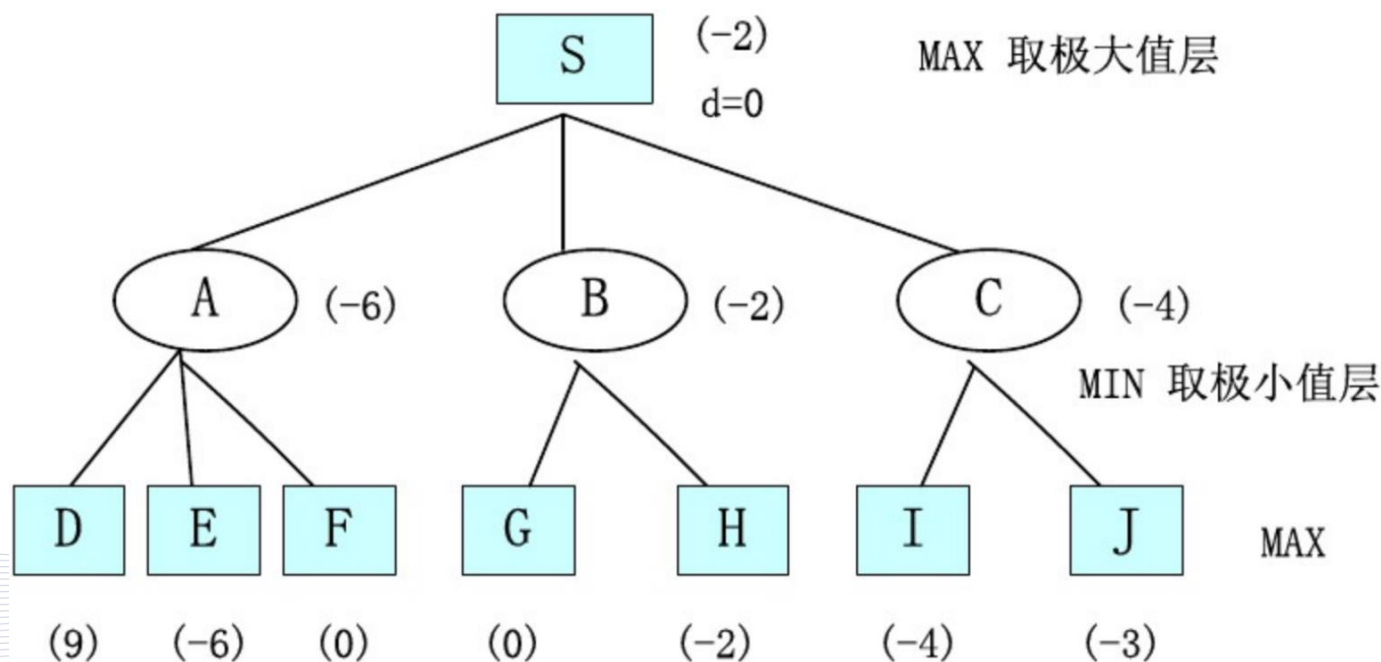
5.2 博弈中的优化决策

- 规定：顶结点深度 $d=0$ ，MAX代表程序方，MIN代表对手方，MAX先走
- 例：一个考虑2步棋的例子



5.2 博弈中的优化决策

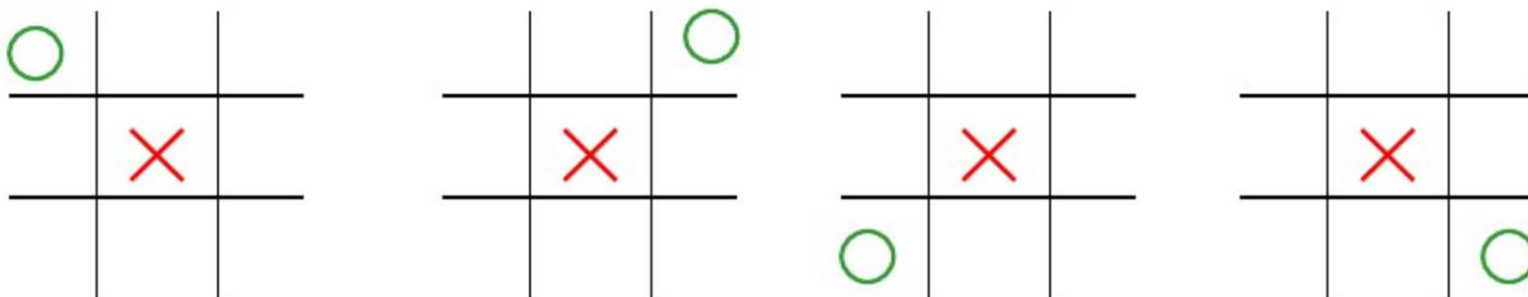
- 规定：顶结点深度 $d=0$ ，MAX代表程序方，MIN代表对手方，MAX先走
- 例：一个考虑2步棋的例子



叶结点给出的数字是用效用函数计算得到的

5.2 博弈中的优化决策

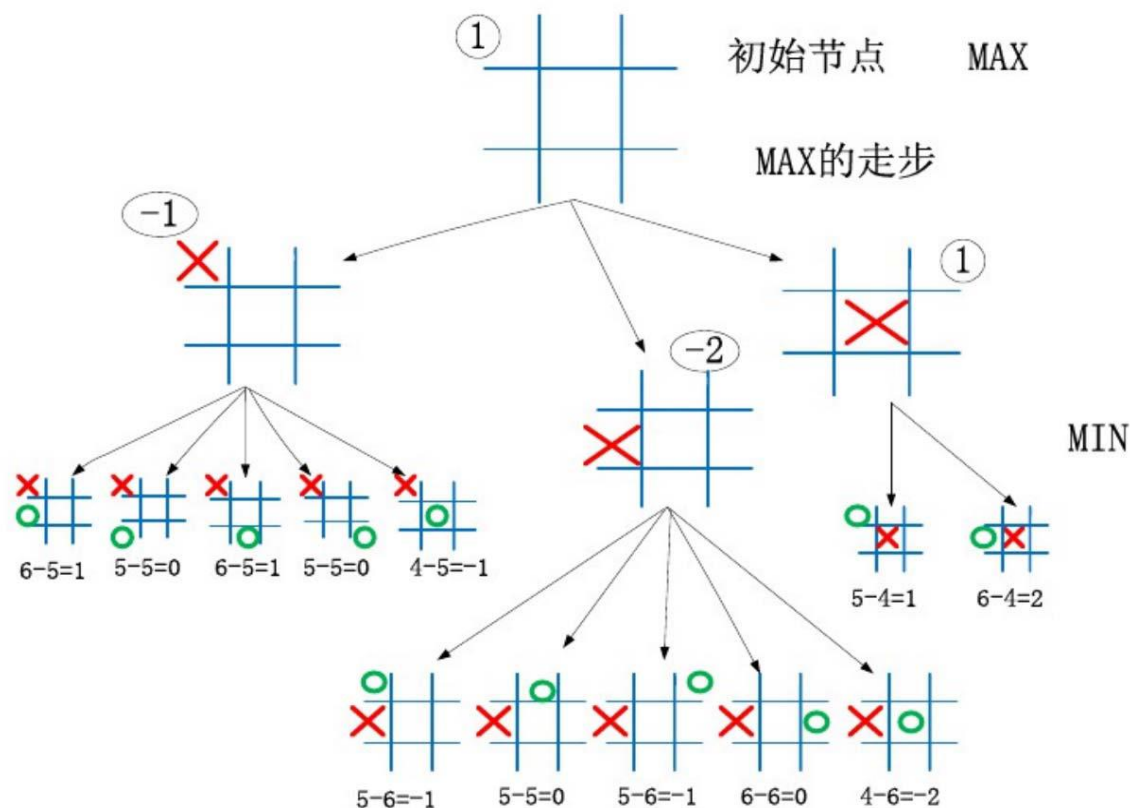
- 例：3x3棋盘的一字棋
- 在搜索过程中，具有对称性的棋局认为是同一棋局，可以大大减少搜索空间。



对称棋局的例子

5.2 博弈中的优化决策

- 例：3x3棋盘的一字棋
- 假定考虑走两步的搜索过程，利用棋盘对称性的条件，则第一次调用算法产生的搜索树为：可以大大减少搜索空间。



■ 例：3x3棋盘的一字棋

- 假设MAX走完第一步后，MAX的对手是在X之上的格子下棋子，这时MAX在新格局下调用算法，第二次产生的搜索树为：



5.2 博弈中的优化决策

■ 多人博弈中的最优决策:

- 考虑非终止状态，考虑图中博弈树上标为X的结点。
- 节点 n 的倒推值是对在该点进行选择的玩家来说效用值最大的后继状态的效用值向量。

轮到移动的玩家

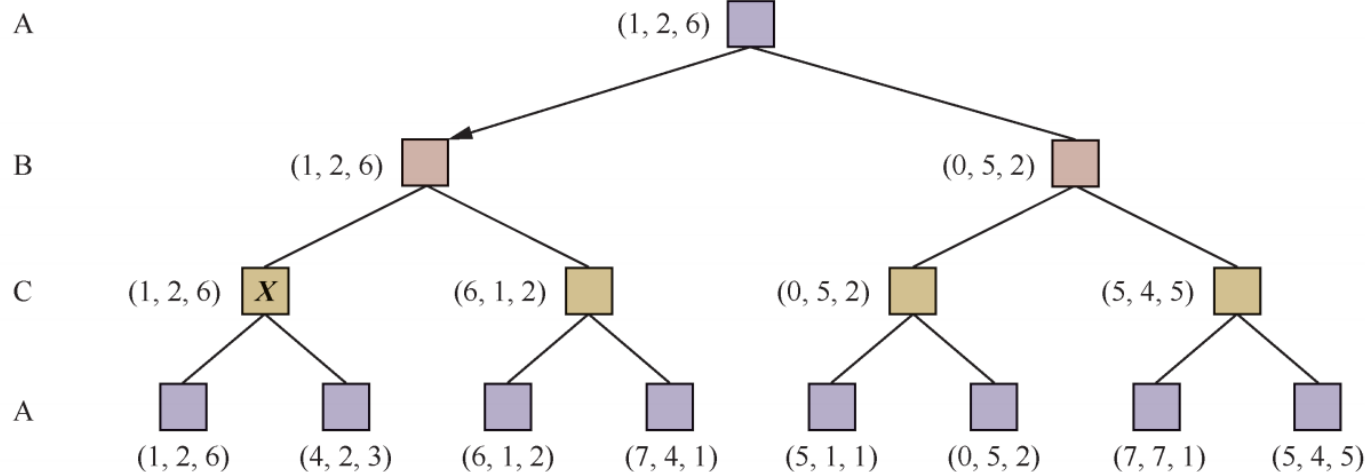
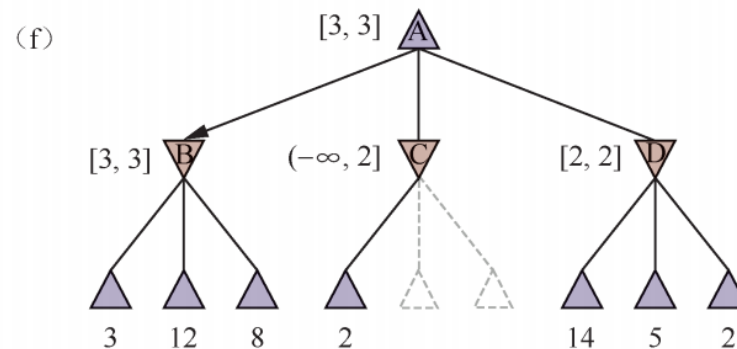
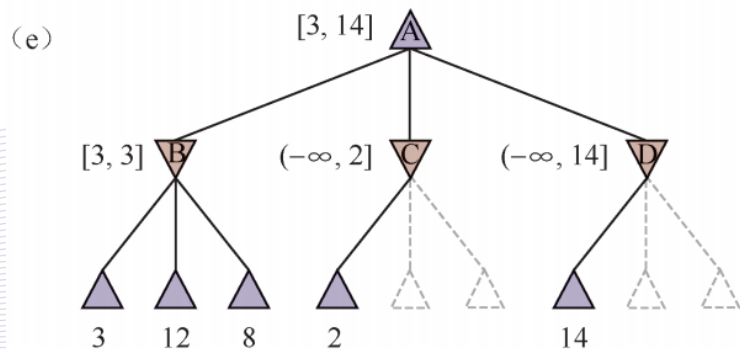
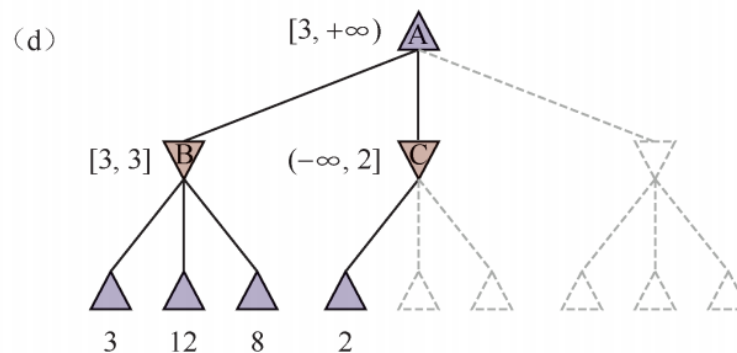
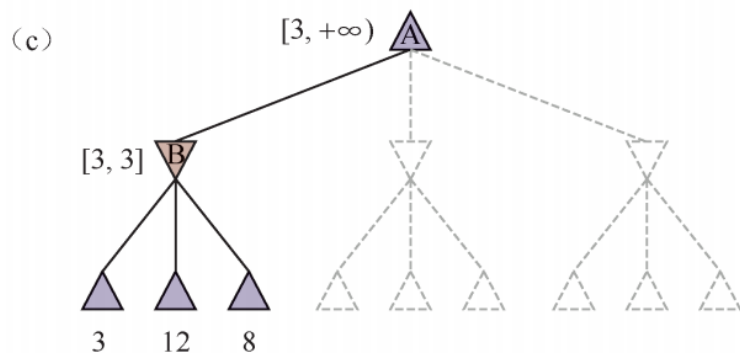
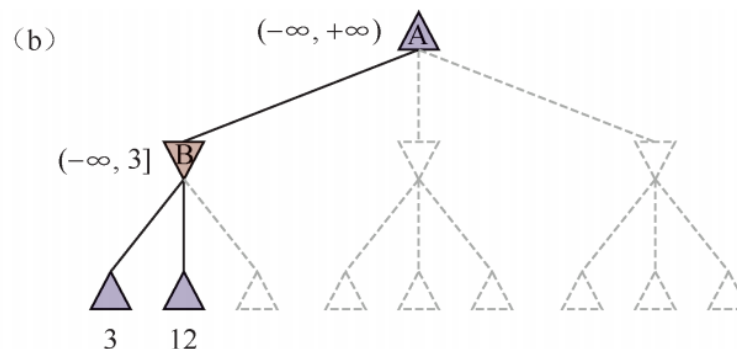
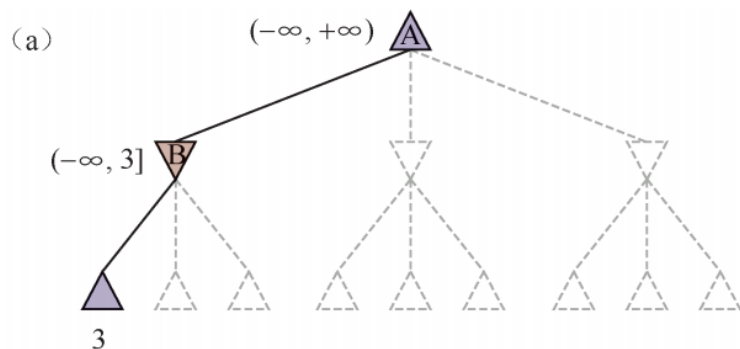


图 5-4 三人博弈的博弈树的前三层，3 个玩家为 A 、 B 、 C 。每个节点都标有 3 个玩家各自的效用值。最佳移动标示在根节点上

- 博弈
- 博弈中的优化决策
- α - β 剪枝
- 不完美的实时决策
- 随机博弈
- 部分可观察的博弈
- 博弈搜索算法的局限性

5.2.3 α - β 剪枝



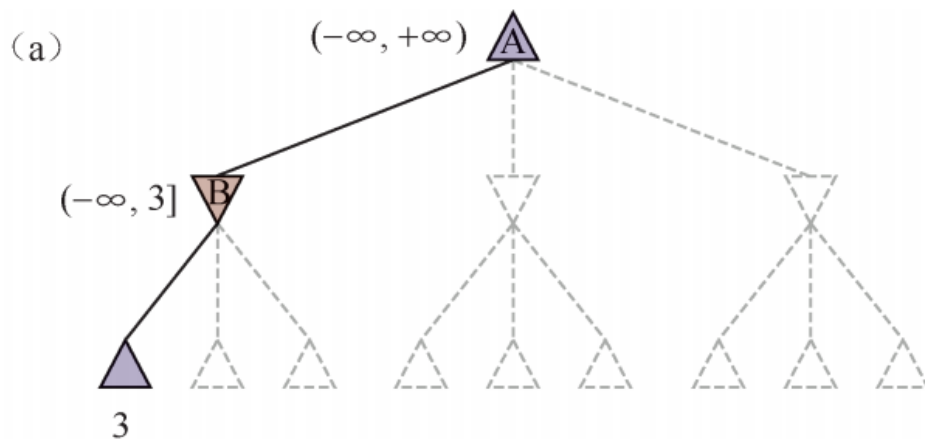
5.2.3 α - β 剪枝

- 将 Minimax 公式简化。
- 节点 n 的倒推值是对在该点进行选择的玩家来说效用值最大的后继状态的效用值向量。

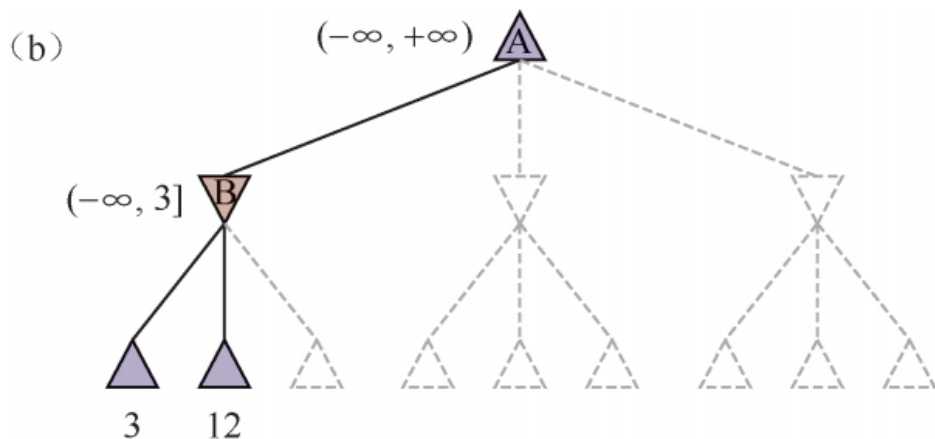
$$\begin{aligned}\text{MINIMAX}(\text{root}) &= \max(\min(3, 12, 8), \min(2, x, y), \min(14, 5, 2)) \\ &= \max(3, \min(2, x, y), 2) \\ &= \max(3, z, 2) \quad \text{其中 } z = \min(2, x, y) \leq 2 \\ &= 3\end{aligned}$$

- 根节点的值以及极小化极大决策与叶节点 x 和 y 的值无关，因此可以将它们剪枝。

5.2.3 α - β 剪枝

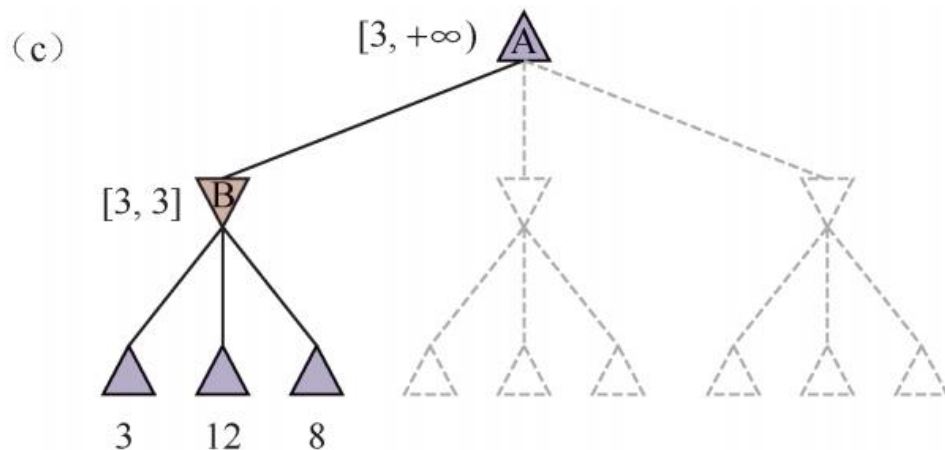


(a) B下面的第一个叶节点值为3。因此，作为min节点，B的值最多为3。

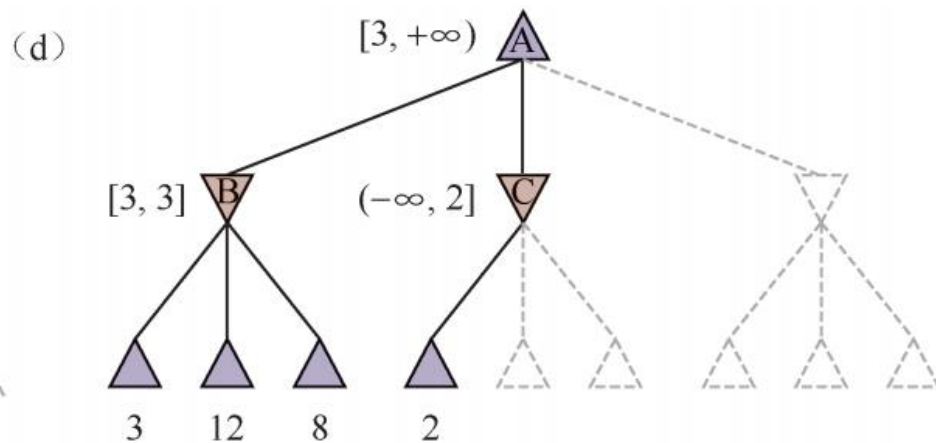


(b) B下面的第二个叶节点值为12，min将避免移动到该节点，所以B的值仍然最多为3。

5.2.3 α - β 剪枝

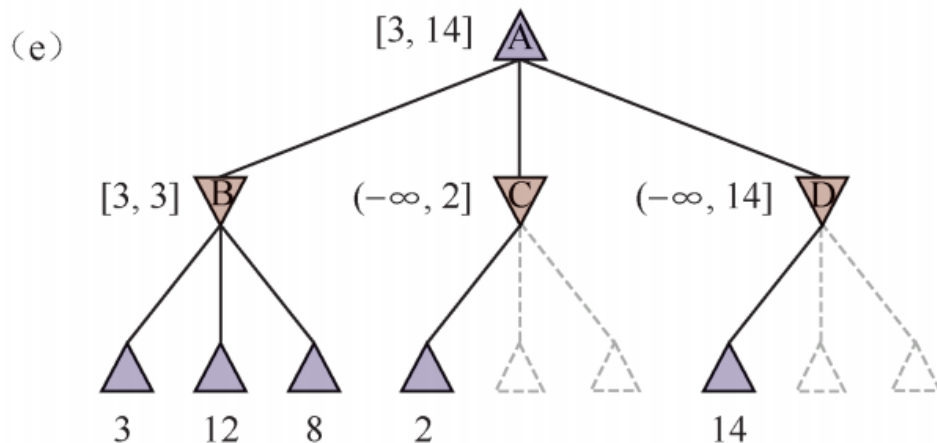


(c) B下面的第三个叶节点值为8，此时我们已经检查完了B的所有后继状态，所以B的值就是3。现在我们可以推断根节点的值至少是3，因为max在根节点处有值为3的选择。

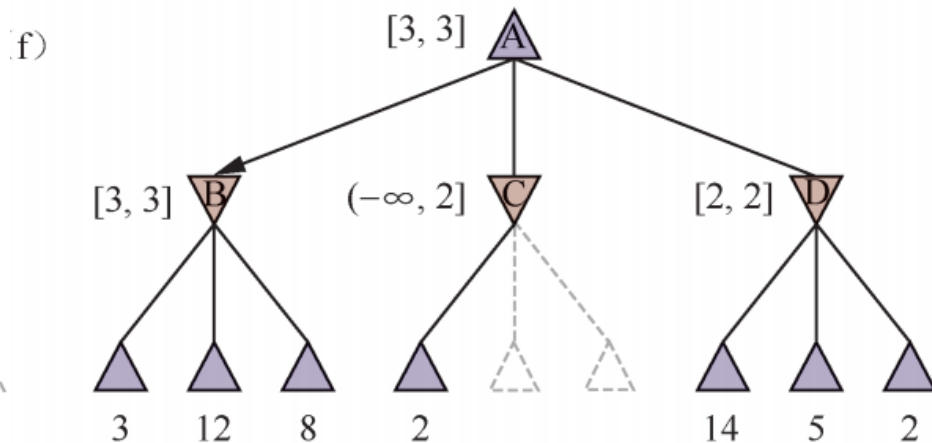


(b) C下面的第一个叶节点值为2。因此，作为min节点，C的值最多为2。但是我们知道B的值为3，所以max永远不会选择C。因此，没有必要再去检查C的其他后继状态。这是 α - β 剪枝的一个实例。

5.2.3 α - β 剪枝



(e) D下面的第一个叶节点值为14，所以D的值最多为14。这仍然高于max的最佳选择（即3），所以我们需要继续探索D的后继状态。注意，此时根节点的所有后继都有界，所以根节点的值也最多为14。



(f) D的第二个后继值为5，所以我们需要继续探索。第三个后继值为2，所以D的值就是2。最终，max在根节点处的决策是移动到值为3的节点B。

5.2.3 α - β 剪枝

- 把**博弈树**和**倒推估值**结合起来进行，再根据一定的条件判定，有可能尽早修剪掉一些无用的分枝。
- 为了使生成和估值过程紧密结合，采用**有界深度优先策略**进行搜索。
- 当生成达到规定深度的结点时，就立即计算效用函数，而一旦某个非端结点有条件确定其倒推值时就立即计算赋值。

5.2.3 α - β 剪枝

- α =到目前为止，路径上发现的max的任一选择点中最佳（即最大值）选择的值。也就是说， α = “至少”。
- β =到目前为止，路径上发现的 min 的任一选择点中最佳（即最小值）选择的值。也就是说， β = “至多”。
- α - β 搜索不断更新 α 和 β 的值，并且一旦当前节点的值比此时的 α （对于max）或 β （对于min）值更差，就剪掉该节点的剩余分支（即终止递归调用）。

5.2.3 α - β 剪枝

function ALPHA-BETA-SEARCH(*game*, *state*) **returns** 一个动作

player $\leftarrow$ game.TO-MOVE(*state*)

value, move $\leftarrow$ MAX-VALUE(*game*, *state*, $-\infty$, $+\infty$)

return move

function MAX-VALUE(*game*, *state*, α , β) **returns** 一个(*utility*, *move*)对

if game.IS-TERMINAL(*state*) **then return** game.UTILITY(*state*, player), null

v, move $\leftarrow -\infty$, null

for each a **in** game.ACTIONS(*state*) **do**

v2, a2 $\leftarrow$ MIN-VALUE(*game*, game.RESULT(*state*, a), α , β)

if v2 > v **then**

v, move $\leftarrow$ v2, a

$\alpha \leftarrow$ MAX(α , v)

if v $\geq$ β **then return** v, move

return v, move

function MIN-VALUE(*game*, *state*, α , β) **returns** 一个(*utility*, *move*)对

if game.IS-TERMINAL(*state*) **then return** game.UTILITY(*state*, player), null

v, move $\leftarrow +\infty$, null

for each a **in** game.ACTIONS(*state*) **do**

v2, a2 $\leftarrow$ MAX-VALUE(*game*, game.RESULT(*state*, a), α , β)

if v2 < v **then**

v, move $\leftarrow$ v2, a

$\beta \leftarrow$ ~~Min~~ (β , v)

if v $\leq$ α **then return** v, move

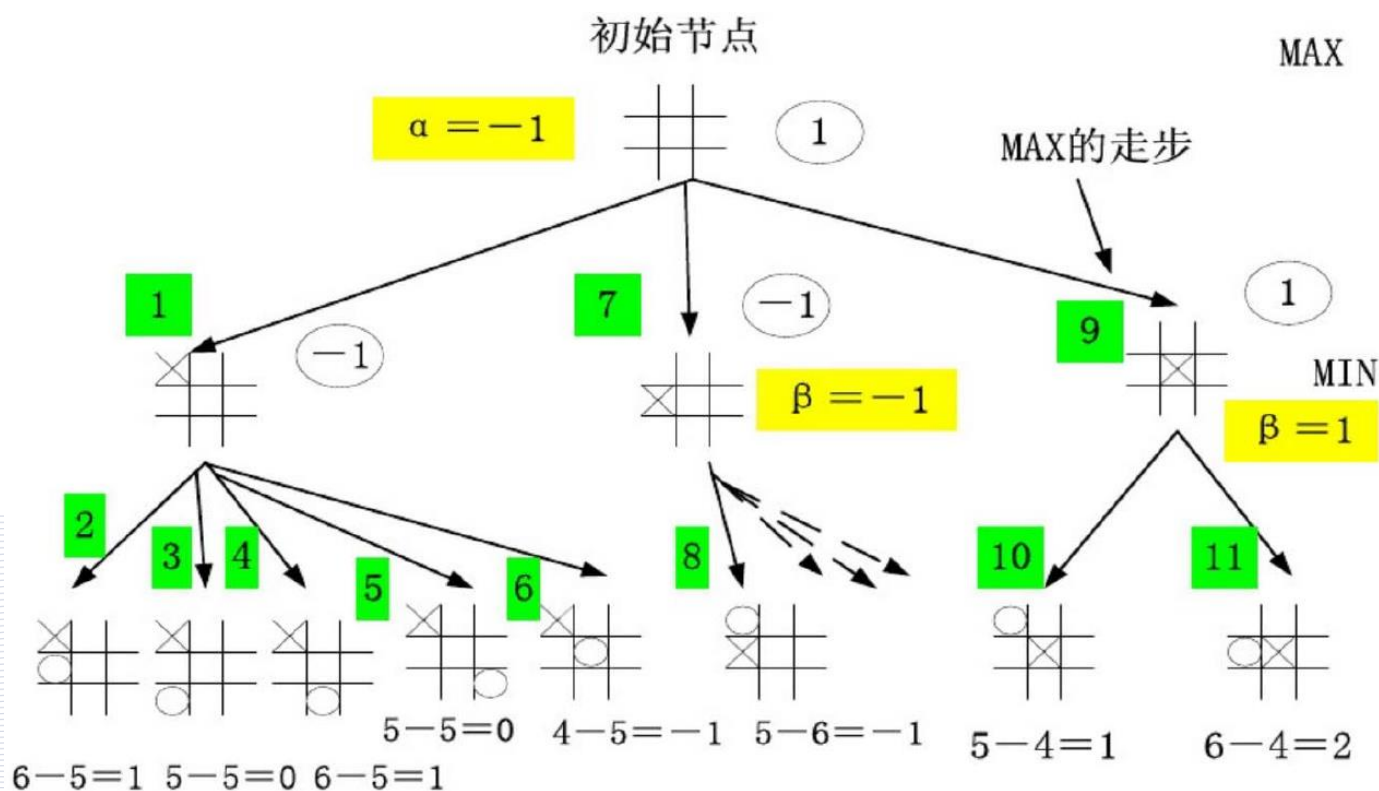
return v, move

图 5-7 α - β 搜索算法。注意，这些函数与图 5-3 中的 MINIMAX-SEARCH 函数相同，除了需要维护变量 α 和 β ，以及在值超出边界时截断搜索

5.2.3 α - β 剪枝

■ 基本思想

- 极大值层的下界值称为 α 。
- 极小值层的上界值称为 β 。



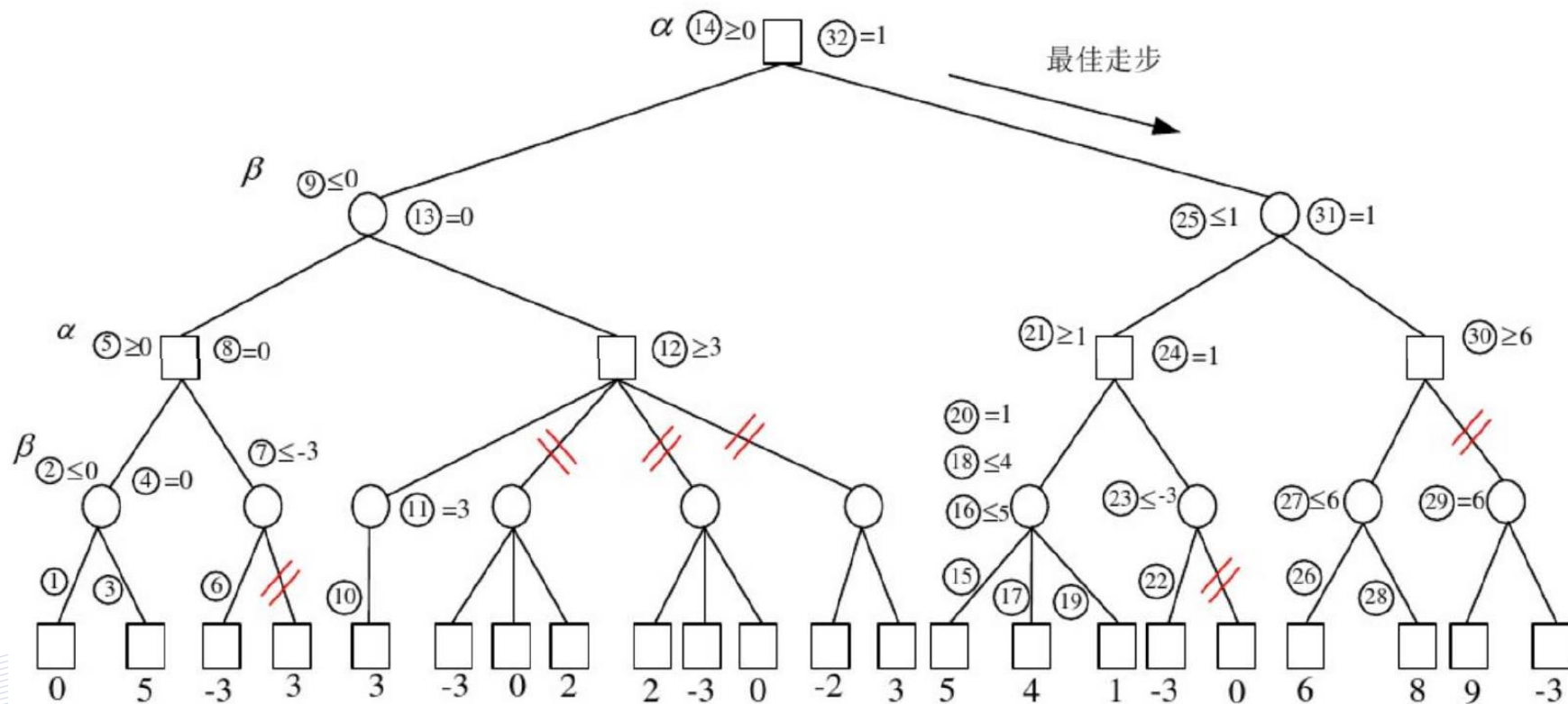
5.2.3 α - β 剪枝

■ 减枝规则

- **α 剪枝**: 若任一极小值层结点的 β 值小于或等于它一先辈极大值层结点的 α 值, 即 $\alpha(\text{先辈层}) \geq \beta(\text{后继层})$, 则可中止该极小值层中这个MIN结点以下的搜索过程。这个MIN结点最终的倒推值就确定为这个 β 值。
- **β 剪枝**: 若任一极大值层结点的 α 值大于或等于它任一先辈极小值层结点的 β 值, 即 $\alpha(\text{后继层}) \geq \beta(\text{先辈层})$, 则可以中止该极大值层中这个MAX结点以下的搜索过程。这个MAX结点的最终倒推值就确定为这个 α 值。

5.2.3 α - β 剪枝

■ 例子



5.2.3 α - β 剪枝

■ 减枝的效率问题

- 比较后得出**最佳 α - β 搜索技术**所生成深度为D处的端结点数约等于不用 α - β 搜索技术所生成**深度为D/2的端结点数**。
- 这就是说，在一般条件下使用 α - β 搜索技术，在同样的资源限制下，可以向前考虑更多的走步数，这样选取当前的最好优先走步，将带来更大的取胜优势。

- 博弈
- 博弈中的优化决策
- α - β 剪枝
- 不完美的实时决策
- 随机博弈
- 部分可观察的博弈
- 博弈搜索算法的局限性

5.3 不完美的实时决策

- 启发式极小化极大值， s 为状态， d 为最大深度。

$$H\text{-MINIMAX}(s, d) = \begin{cases} \text{EVAL}(s, \text{MAX}) & \text{如果 } \text{Is-CUTOFF}(s, d) \\ \max_{a \in \text{Actions}(s)} H\text{-MINIMAX}(\text{RESULT}(s, a), d + 1) & \text{如果 } \text{To-Move}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} H\text{-MINIMAX}(\text{RESULT}(s, a), d + 1) & \text{如果 } \text{To-Move}(s) = \text{MIN} \end{cases}$$

- 回顾极小化极大值

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{如果 } \text{Is-TERMINAL}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{如果 } \text{To-Move}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{如果 } \text{To-Move}(s) = \text{MIN} \end{cases}$$

5.3 不完美的实时决策

- 在实践中，这种方法需要分析太多类别，因此需要非常多的经验去估计所有的可能性。
 - 国际象棋中：白兵数目、黑兵数目、白后数目、黑后数目等
 - 两兵对一兵，82%的状态通向胜利，2%导致失败，16%为平局
 - 子力价值：兵值1分，马或象值3分，车值5分，后值9分
- 数学上，这种评价函数称为**加权线性函数**（weighted linear function），可以表示为如下形式：

$$\text{EVAL}(s) = w_1 f_1(s) + w_2 f_2(s) + \cdots + w_n f_n(s) = \sum_{i=1}^n w_i f_i(s)$$

其中， w_i 是权值， f_i 是棋局的某个特征。对于国际象棋， f_i 可能是棋盘上每种棋子的数目， w_i 可能是每种棋子的价值。

5.3 不完美的实时决策

- 线性是不是合理？非线性组合？
- 如何得到特征和权重？
- 机器学习

5.3 不完美的实时决策

■ 截断搜索

- 下一步是修改 Alpha-Beta-Search，让它在合适的时候调用启发式 Eval 函数截断搜索。
- 将图5-7中Is-Terminal的两行代码替换为下面这行：

if *game.Is-CUTOFF(state, depth)* **then return** *game.EVAL(state, player), null*

function MAX-VALUE(*game, state, α , β*) **returns** 一个(*utility, move*)对

```

if game.Is-TERMINAL(state) then return game.UTILITY(state, player), null
v, move  $\leftarrow -\infty, null$ 
for each a in game.ACTIONS(state) do
  v2, a2  $\leftarrow$  MIN-VALUE(game, game.RESULT(state, a),  $\alpha$ ,  $\beta$ )
  if v2 > v then
    v, move  $\leftarrow$  v2, a
     $\alpha \leftarrow$  MAX( $\alpha$ , v)
  if v  $\geq \beta$  then return v, move
return v, move
  
```

function MIN-VALUE(*game, state, α , β*) **returns** 一个(*utility, move*)对

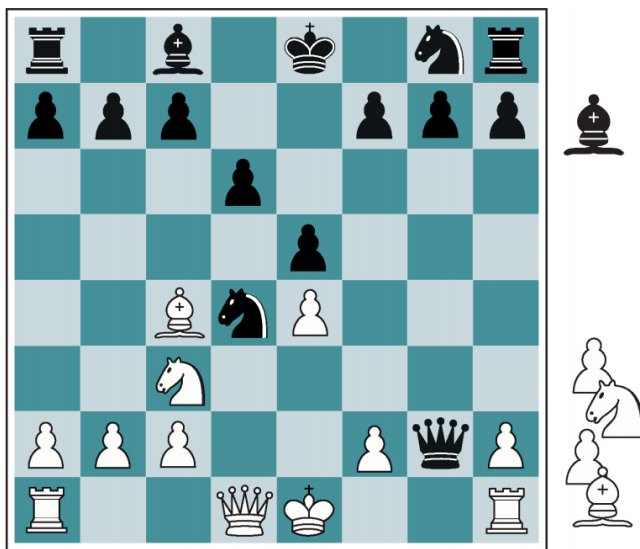
```

if game.Is-TERMINAL(state) then return game.UTILITY(state, player), null
v, move  $\leftarrow +\infty, null$ 
for each a in game.ACTIONS(state) do
  v2, a2  $\leftarrow$  MAX-VALUE(game, game.RESULT(state, a),  $\alpha$ ,  $\beta$ )
  if v2 < v then
    v, move  $\leftarrow$  v2, a
     $\beta \leftarrow$  MAX( $\beta$ , v)
  if v  $\leq \alpha$  then return v, move
return v, move
  
```

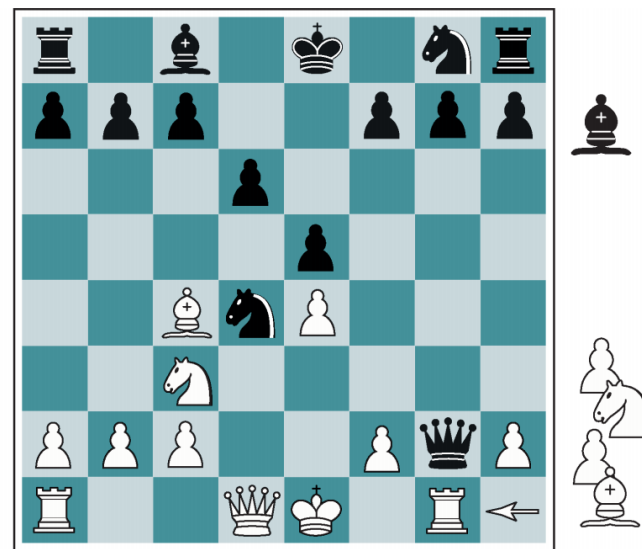
5.3 不完美的实时决策

■ 截断搜索

- 假设棋局到达了深度限制，此时黑方有一马两兵的优势，根据这个状态的启发式函数值，认为这个状态导致黑马获胜
- 其实下一步白方可以毫无意外地吃掉黑方皇后因此这个棋局是白棋赢，需要向前多看一步才能预测。



(a) 轮到白棋移动



(b) 轮到白棋移动

5.3 不完美的实时决策

■ 截断搜索

□ 评估函数只适用于那些静态局面

- 这些局面中不存在会使评估值大幅度摇摆变化的待定移动（例如吃掉皇后）

□ 非静态的棋局可以继续搜索直到达到静态局面，这种额外的搜索称为静态搜索

- 有时会被进一步限制为只考虑特定类型的移动（例如吃子），它能快速消除当前局面的不确定性。

5.3 不完美的实时决策

■ 向前剪枝

- 若在最新的个人电脑上可每秒生成和评估约一百万个结点，则允许我们在标准的时间控制下(每步棋3分钟)对每步棋可搜索约2亿个结点
 - 国际象棋的分枝因子平均为35， 35^5 约为5000万
 - 如果使用极小极大搜索，在大约1分钟的计算时间内只能向前搜索5层，很容易被平均水平的人类棋手欺骗。
 - 如果使用alpha-beta搜索和大型换位表，可以预测大约14层，已经到达了专家级水平。

- 博弈
- 博弈中的优化决策
- α - β 剪枝
- 不完美的实时决策
- 随机博弈
- 部分可观察的博弈
- 博弈搜索算法的局限性

5.5 随机博弈

- 若现实生活中，很多不可预知的外部事件会把我们推到无法预计的情景中
- 随机博弈：许多博弈用随机因素，如掷骰子，来反映这种不可预测性
 - 西洋双陆棋就是运气和技巧相组合的游戏。

5.5 随机博弈

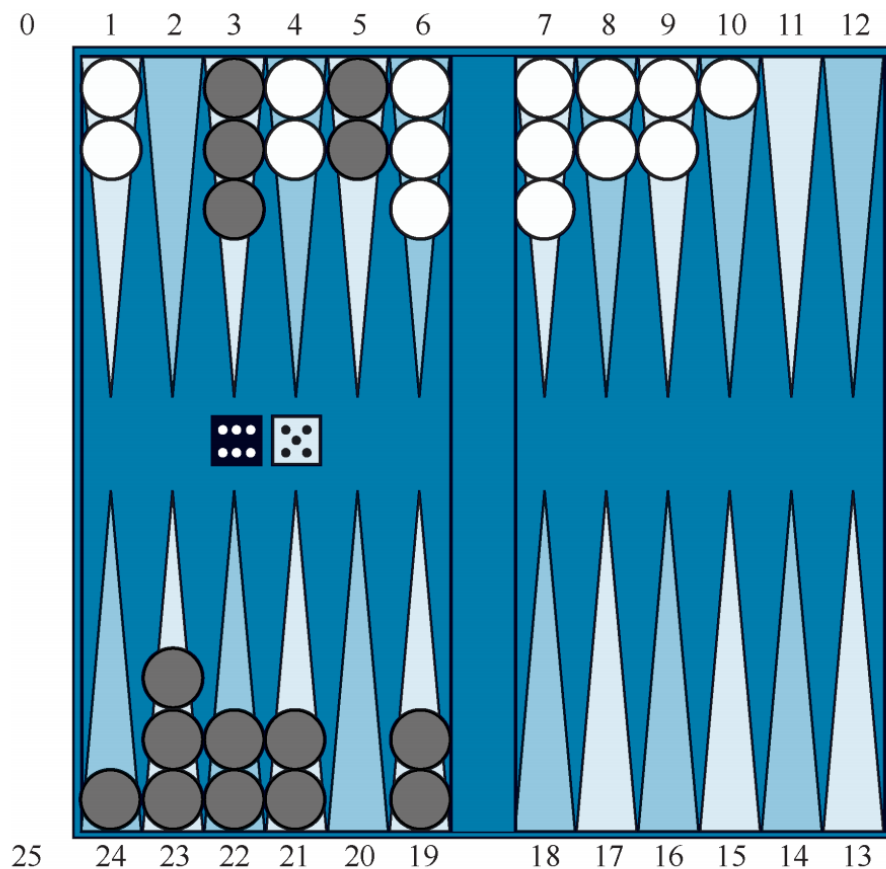


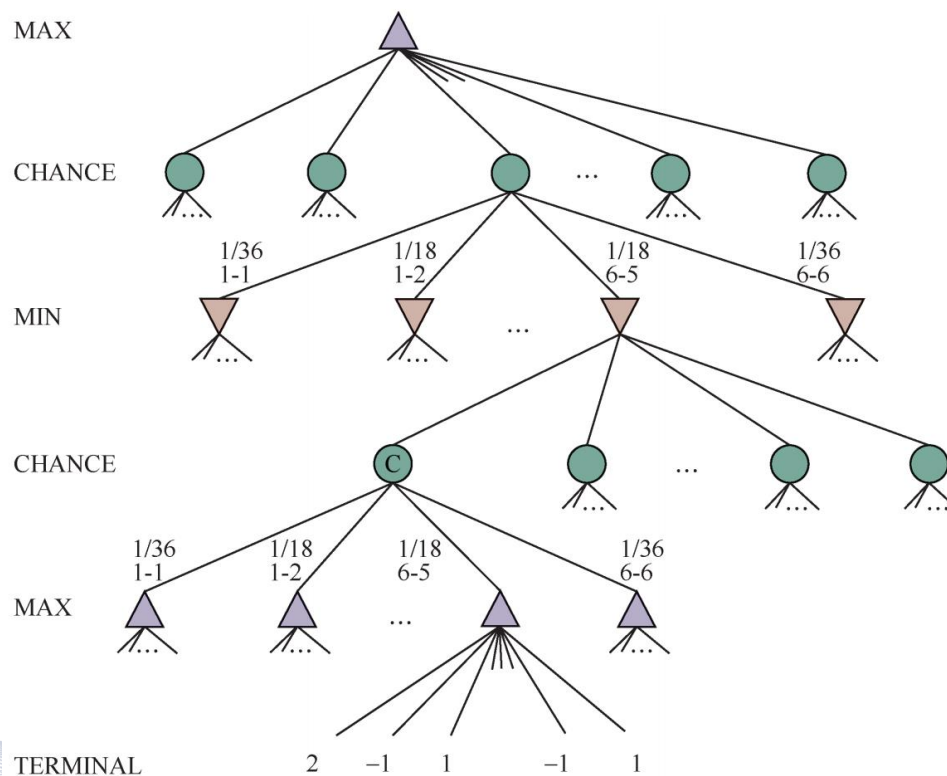
图 5-12 一个典型的西洋双陆棋局面。游戏的目标是把自己的所有棋子移出棋盘。黑方向 25 顺时针移动，白方向 0 逆时针移动。一个棋子可以移动到任何位置，除非那里有多个对方棋子；如果只有一个对方棋子，对方棋子就会被吃掉，然后必须从起点重新开始。图中所示的局面，黑棋已经掷出了 6-5，必须从 (5-11, 5-10)、(5-11, 19-24)、(5-10, 10-16) 和 (5-11, 11-16) 这 4 种合法移动中选择，其中符号 (5-11, 11-16) 表示将一个棋子从位置 5 移动到位置 11，另一个棋子从位置 11 移动到位置 16

5.5 随机博弈

□ 博弈树除了max和min节点外，还必须包括机会节点。

- 从每个机会节点引出的分支表示可能掷出的骰子点数，每个分支都标有掷出的点数及其概率

- 两个骰子总共21种骰法，其中：
- 6个有相同骰子数的组合 (**1/36概率**)
- 15个不同骰子数的组合 (**1/18概率**)



5.5 随机博弈

■ 极小化极大值 → 期望极小化极大值

- 将确定性博弈的极小化极大值推广为包含机会节点的博弈的期望极小化极大值。

EXPECTIMINIMAX(s) =

$$\begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{如果 Is-TERMINAL}(s) \\ \max_a \text{EXPECTIMINIMAX}(\text{RESULT}(s, a)) & \text{如果 To-MOVE}(s) = \text{MAX} \\ \min_a \text{EXPECTIMINIMAX}(\text{RESULT}(s, a)) & \text{如果 To-MOVE}(s) = \text{MIN} \\ \sum_r P(r) \text{EXPECTIMINIMAX}(\text{RESULT}(s, r)) & \text{如果 To-MOVE}(s) = \text{CHANCE} \end{cases}$$

r 表示可能的掷骰子结果（或其他概率事件），而 $\text{Result}(s, r)$ 仍表示状态 s ，附加了掷骰子结果 r 。

5.5 随机博弈

■ 机会博弈的评价函数

- 如果更改一些评估值，即使优先顺序保持不变，程序的选择也会完全不同。
- 评价函数应该返回获胜概率（对于结果非输或赢的博弈返回的是期望效用值）的正线性变换值。

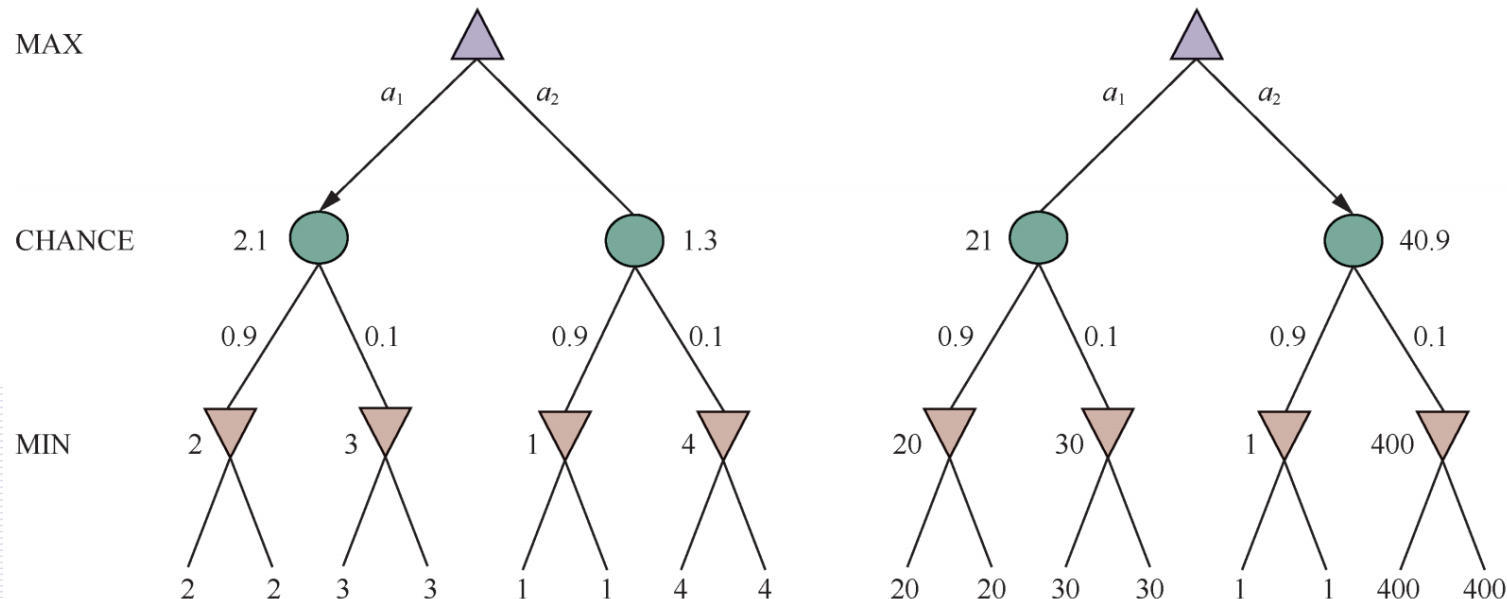


图 5-14 在保持叶节点值排序不变的情况下，不同的叶节点赋值改变了最佳移动

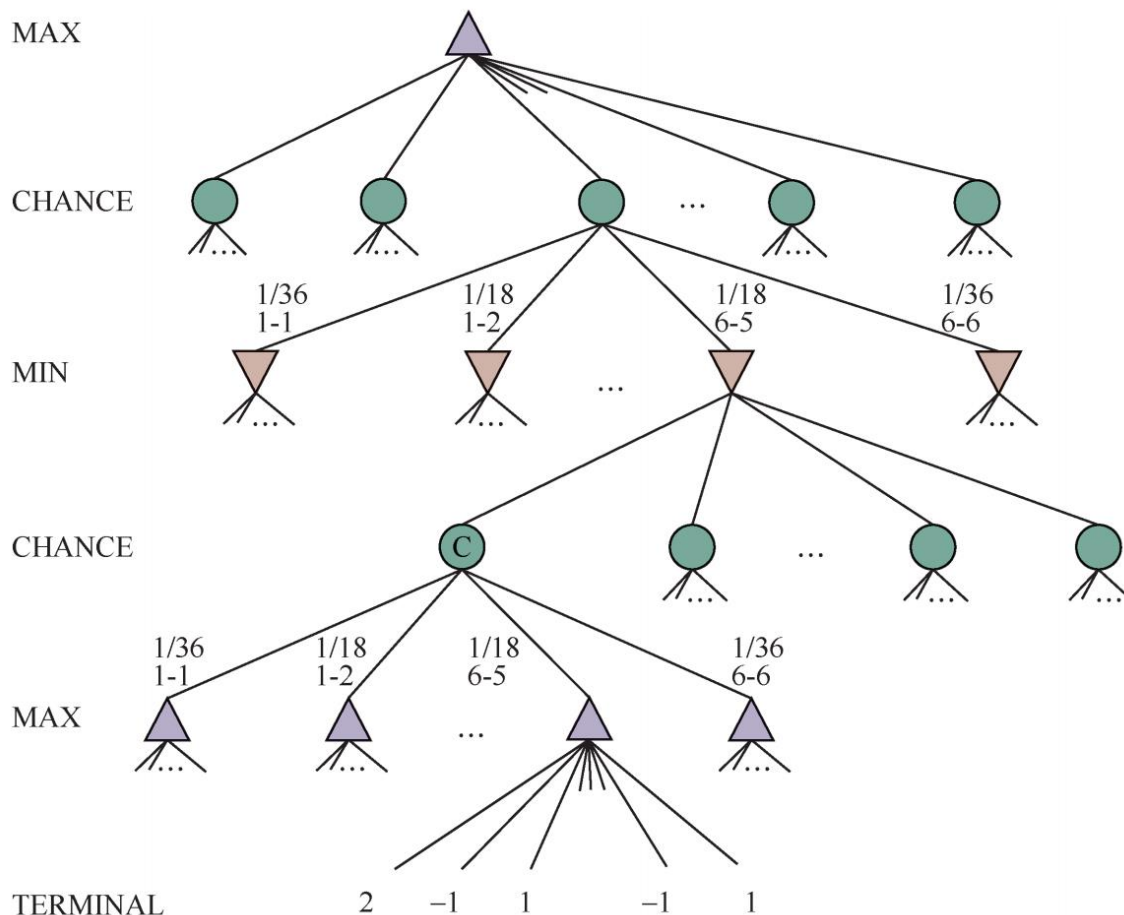
5.5 随机博弈

■ 期望极小极大值的复杂度

- 算法的时间复杂度为 $O(b^m n^m)$ ，其中 n 是**掷骰子不同结果的数目**。
- 由于额外代价的存在使得在大多数机会博弈中向前看很远是不现实的。
 - 西洋双陆棋中， n 是21， b 通常是20左右
- 可以对机会博弈树适用类似 α - β 剪枝技术？
 - 限制**效用函数的取值范围**→不用检查机会结点的所有子结点就可以设置机会结点的上界

5.5 随机博弈

■ 例子



- 考虑结点C，是否有可能在考察它的全部子结点前就发现C的上界。
- 通过限制效用函数的可能值的范围实现。

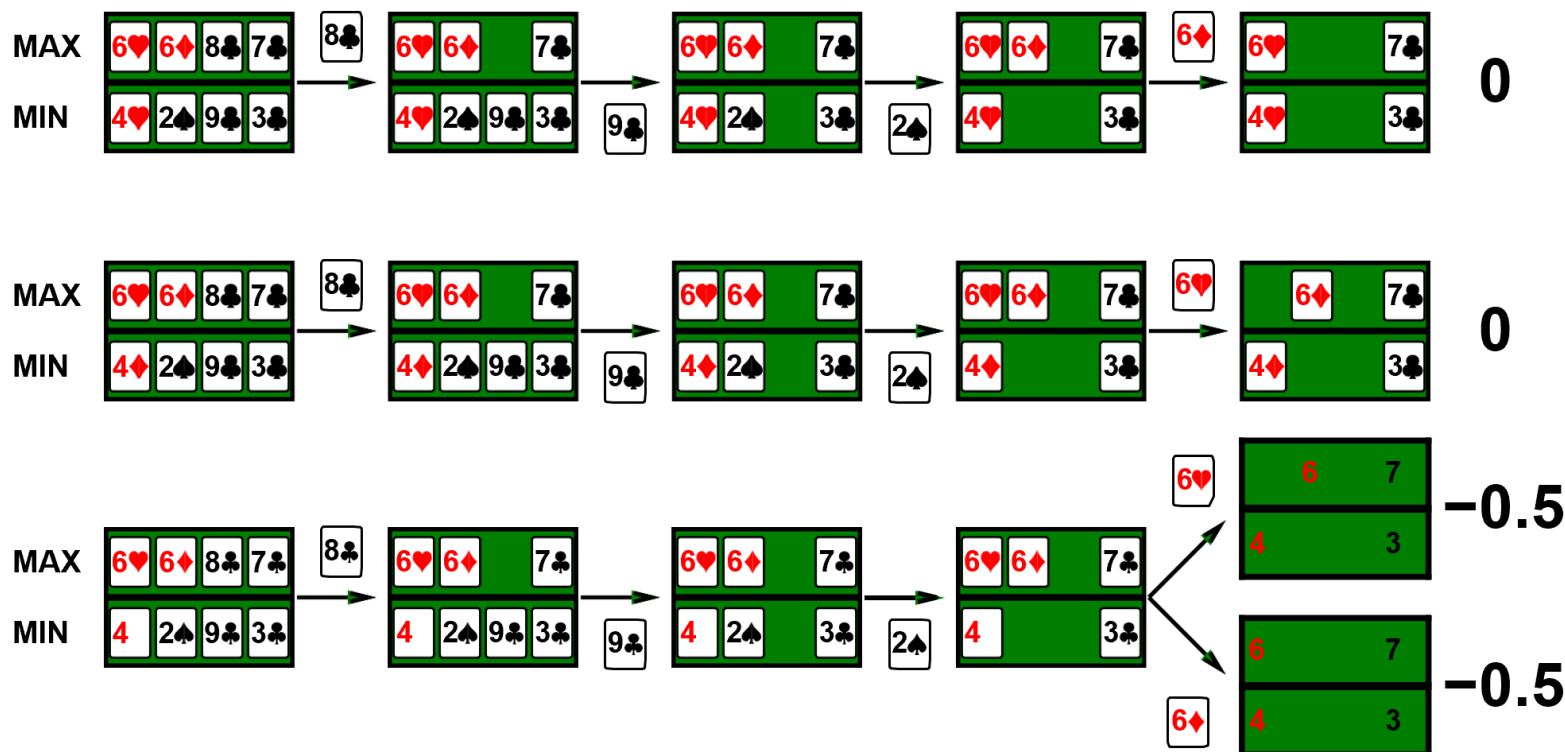
- 博弈
- 博弈中的优化决策
- α - β 剪枝
- 不完美的实时决策
- 随机博弈
- 部分可观察的博弈
- 博弈搜索算法的局限性

5.6 部分可观测博弈

- 国际象棋就是战争，但国际象棋缺少真实战争的一个主要特征——部分可观测性。
 - 敌方的存在和部署通常是不知道的
 - 战争中通常会有侦察兵和间谍收集信息
 - 隐瞒信息或虚张声势搞乱敌方
- 军棋：国际象棋的部分可观察变种，棋子可以移动但是对方看不见棋子是什么。
- 牌类游戏：发牌是随机的，选手拿的牌别人无法知道。

5.6 部分可观测博弈

Four-card bridge/whist/hearts hand, Max to play first



5.6 部分可观测博弈

■ 牌类游戏

- 随机算法，考虑未知牌的所有可能应对策略。假设应对策略s发生的概率是P(s)

$$\operatorname{argmax}_a \sum_s P(s) \operatorname{MINIMAX}(\operatorname{RESULT}(s, a))$$

- 大多数牌类游戏，可能的应对策略数目相当大使用蒙特卡洛近似:不考虑所有可能组合，随机采样N个样本，设组合s在样本中出现的概率与P(s)成正比

$$\operatorname{argmax}_a \frac{1}{N} \sum_{i=1}^N \operatorname{MINIMAX}(\operatorname{RESULT}(s_i, a))$$

5.6 部分可观测博弈

■ 有意思的例子

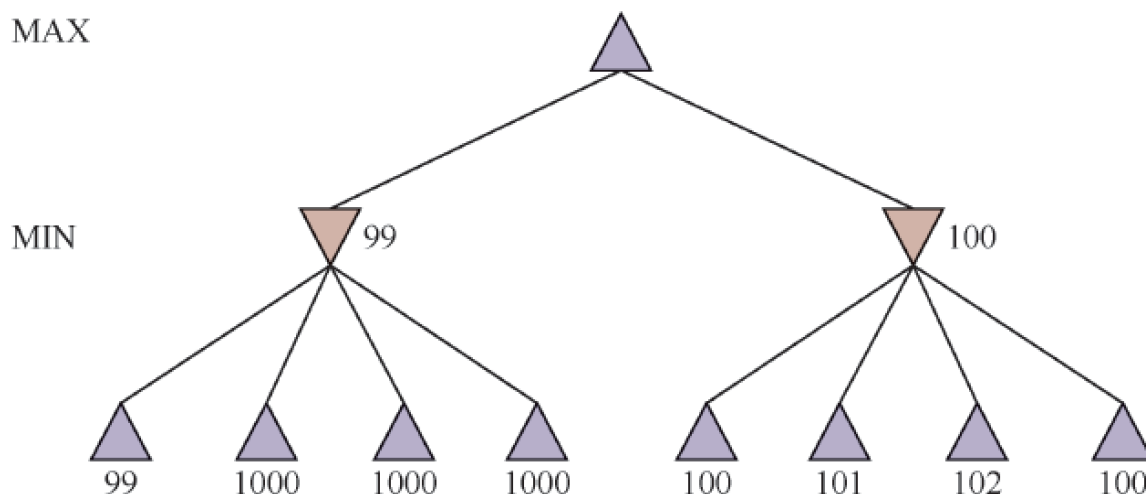
- 第一天：道路A通向一桶金子，道路B通向一个岔路口。你可以看到，岔路口左转是两桶金子，右转则会撞上一辆公共汽车。
- 第二天：道路A通向一桶金子，道路B通向一个岔路口。你可以看到，岔路口右转是两桶金子，左转则会撞上一辆公共汽车。
- 第三天：道路A通向一桶金子，道路B通向一个岔路口。你知道岔路口的一个分支通向两桶金子，另一个分支会撞上一辆公共汽车。遗憾地是，你不知道哪个分支通向金子。

第一天，B是正确选择；第二天，B也是正确选择；第三天，情况和第一天或第二天一样，所以B仍然是正确选择。

- 博弈
- 博弈中的优化决策
- α - β 剪枝
- 不完美的实时决策
- 随机博弈
- 部分可观察的博弈
- 博弈搜索算法的局限性

5.7 博弈搜索算法的局限性

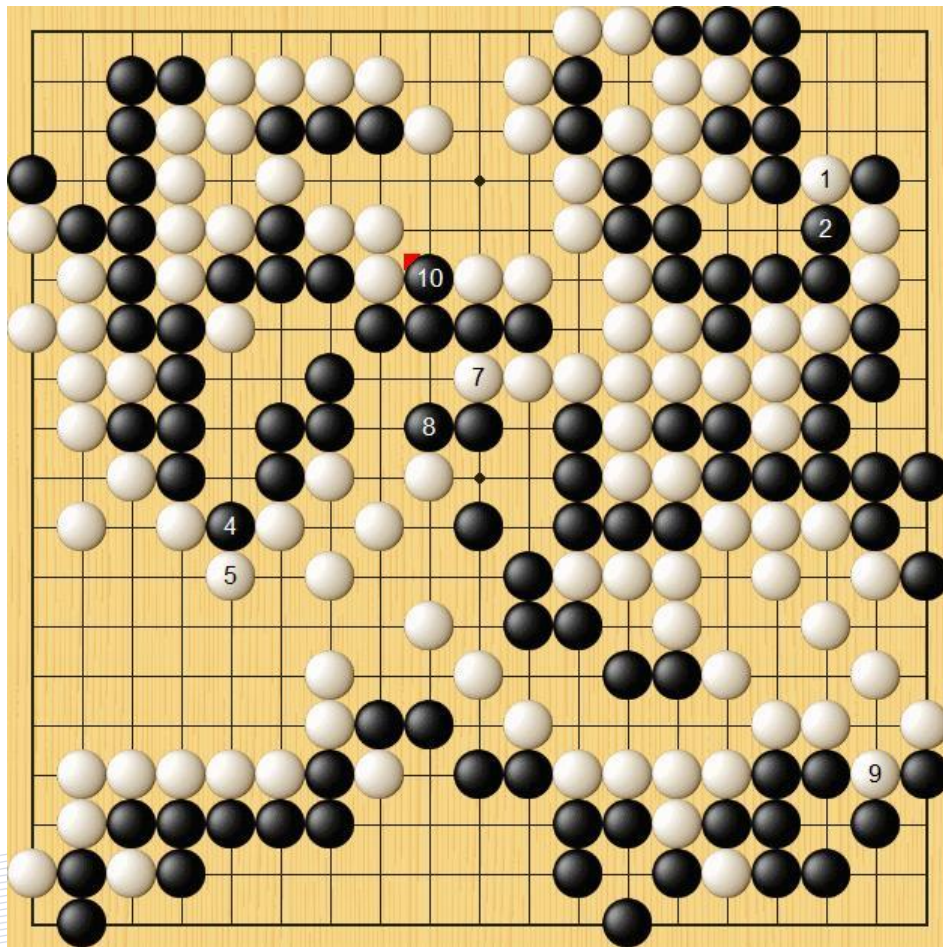
- 计算复杂博弈的最优决策是非常困难的，所有算法都必须做出一些假设和近似
 - α - β 搜索使用启发式评价函数作为近似
 - 蒙特卡罗搜索计算随机选择的模拟的近似平均值
 - 当分支因子较高或者评价函数难以定义时，首选蒙特卡罗搜索
- α - β 搜索容易受到启发式函数的近似误差的影响



5.7 博弈搜索算法的局限性

- α - β 搜索和蒙特卡罗搜索都是在单步移动的层级上进行推理，而人类可以在更抽象的层次上进行推理
- 机器学习融入博弈搜索过程。早期依靠人类的专业知识人为制定评价函数、开局库、搜索策略和高效技巧。AlphaZero可以自我对弈，自主学习进化非人类在特定游戏上的专业知识

围棋



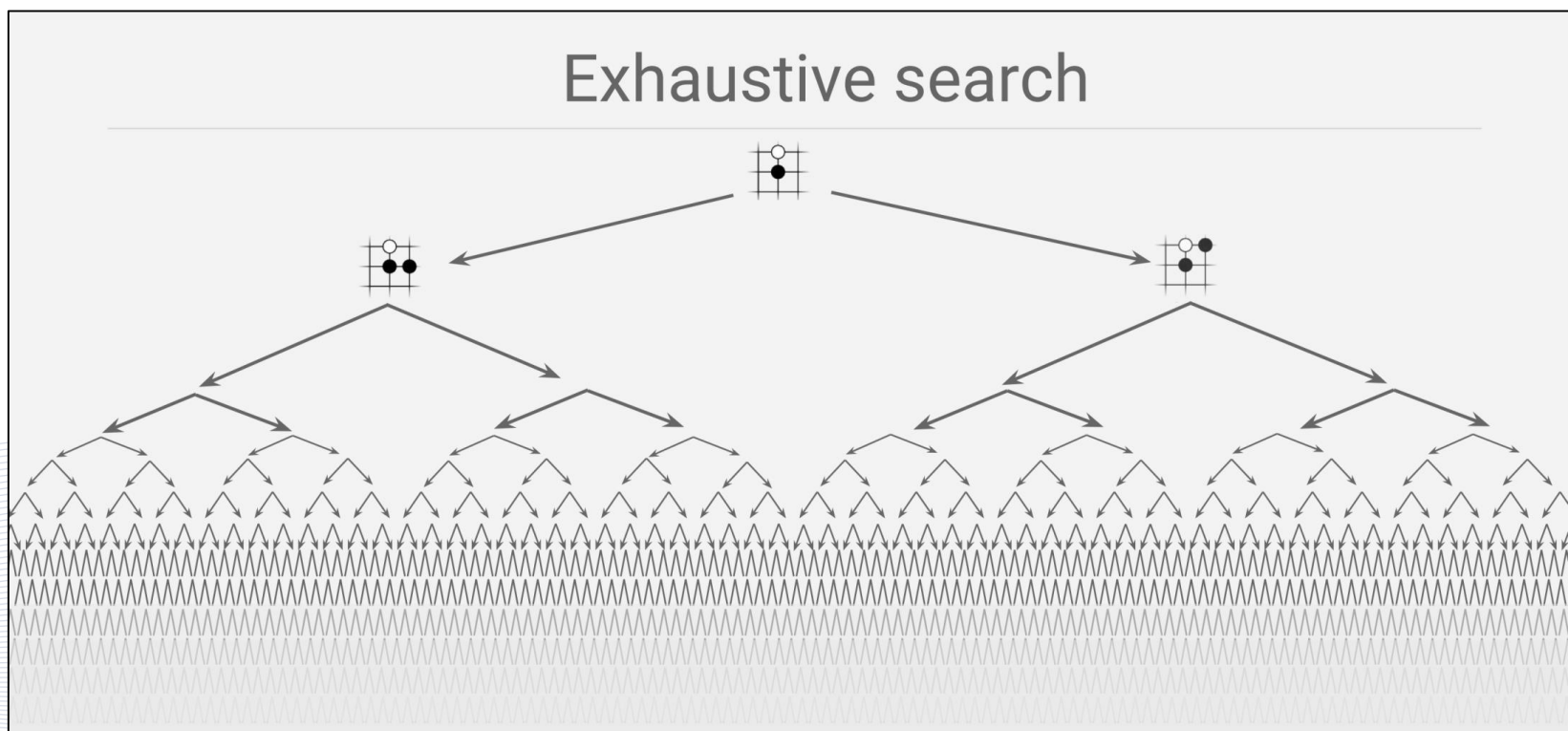
游戏说明

- ◆ 围棋： 一人执黑、一人执白，双方轮流下，围地盘
- ◆ 博弈类型 二人零和完美信息博弈

围棋

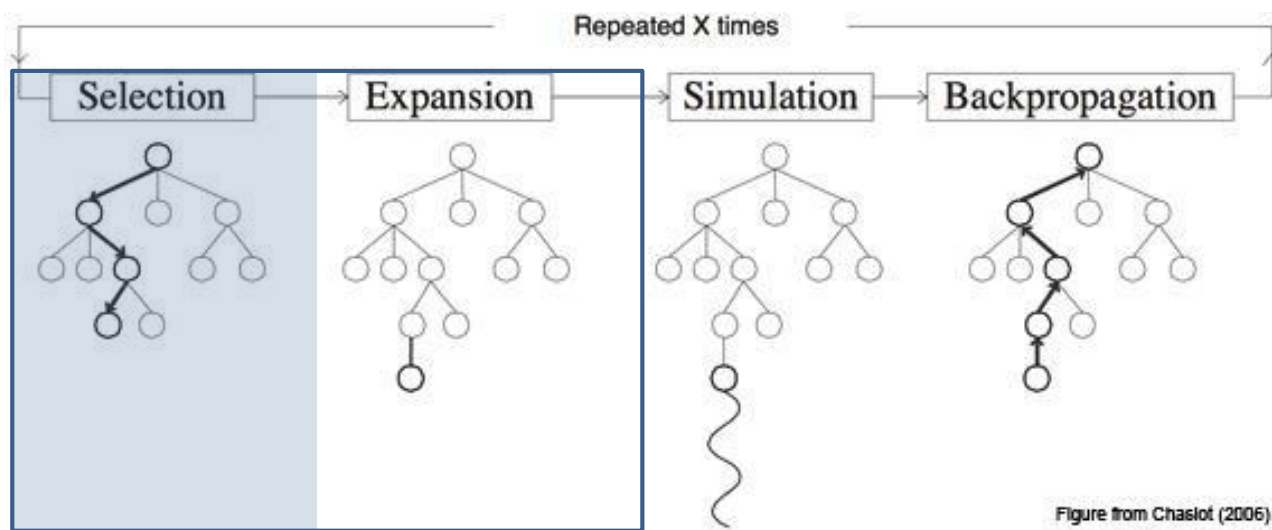
计算复杂度

- 围棋游戏建模：博弈树
- 博弈树复杂度 10^{360} 状态空间复杂度 $\approx 3^{361} \approx 10^{172}$
- 因此采用常规的树搜索算法，比如Minimax搜索方法来求解19路围棋的最优策略，在人类当前的算力下是不可能的



围棋AI求解框架：MCTS

MCTS (Monte Carlo Tree Search)，是一类**概率统计型**树搜索算法的统称，可以较为有效地解决一些探索空间巨大的问题，相比于传统的深度优先或者广度优先算法，MCTS 执行一种非对称的树的适应搜索空间拓扑结构的生长，这个算法会更频繁地访问更有趣的节点，并聚焦其搜索时间在更加相关的树的部分，算法流程一般分为四步：选择、拓展、评估、回传



落子选择

落子评估

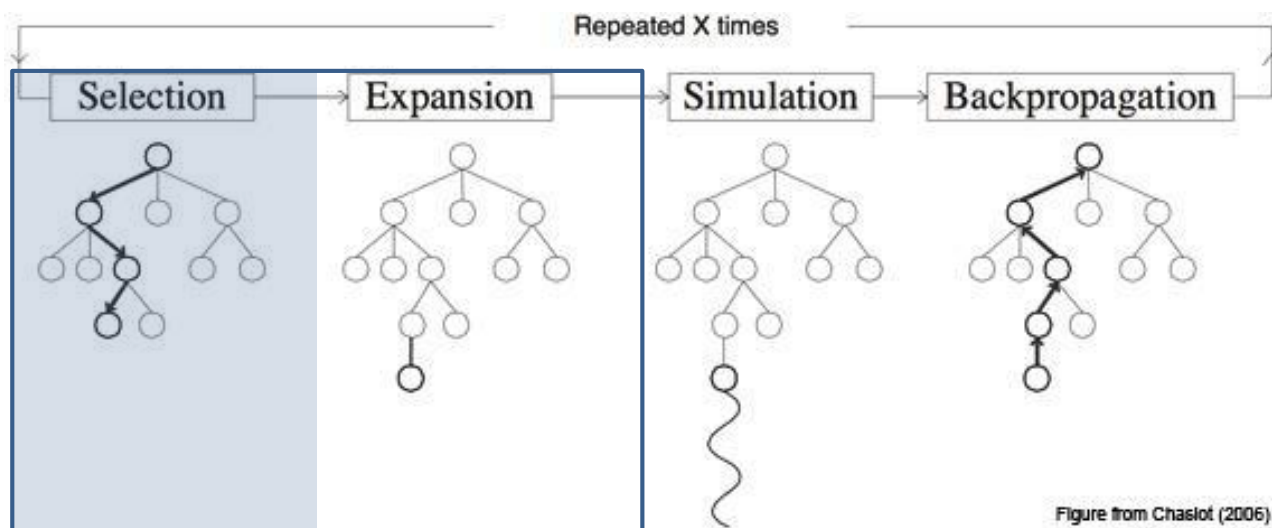
◆ 算法流程

总共对当前状态 S_t 进行K次采样，在每次采样的过程中：

- 选择：这一步会从根节点开始，每次都选一个“最值得搜索的子节点”，可以使用 ϵ -greedy的方法或者UCB方法来选择子节点，直到来到一个“存在未扩展的子节点”的节点。“存在未扩展的子节点”，是指当前构建的博弈树中该节点的子节点还没有加入到树中，如初始时就只能选择到根节点就结束。这时我们进入第二步

围棋AI求解框架：MCTS

MCTS (Monte Carlo Tree Search)，是一类**概率统计型**树搜索算法的统称，可以较为有效地解决一些探索空间巨大的问题，相比于传统的深度优先或者广度优先算法，MCTS 执行一种非对称的树的适应搜索空间拓扑结构的生长，这个算法会更频繁地访问更有趣的节点，并聚焦其搜索时间在更加相关的树的部分，算法流程一般分为四步：选择、拓展、评估、回传



落子选择

落子评估

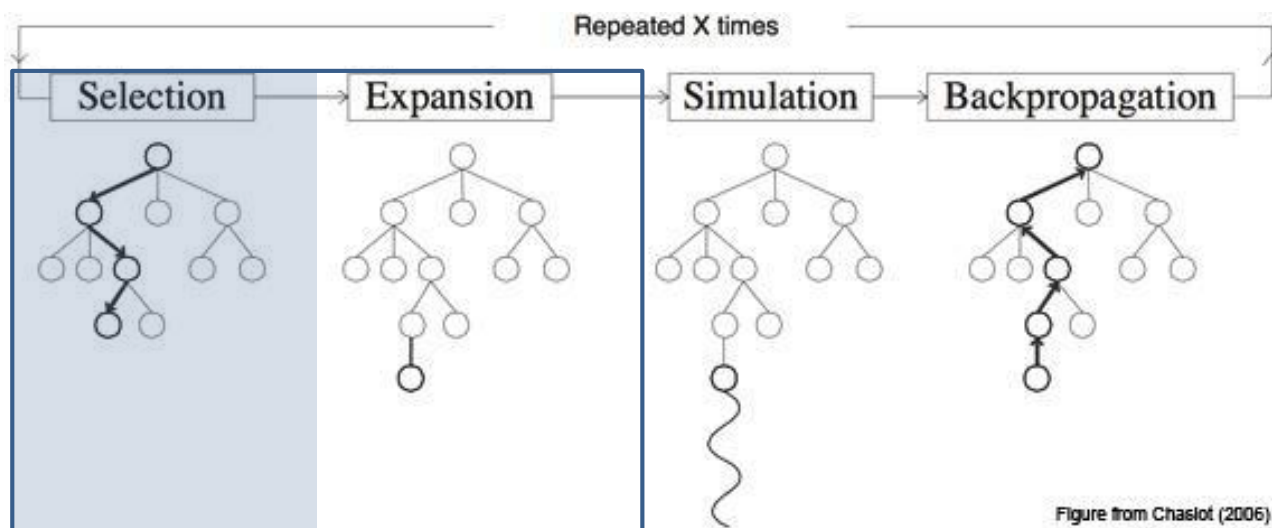
◆ 算法流程

总共对当前状态 S_t 进行K次采样，在每次采样的过程中：

- 拓展：在这个搜索到的存在未扩展子节点的节点，加上一个新的子节点，新加入的子节点的价值和访问次数都是0。这时我们进入第三步

围棋AI求解框架：MCTS

MCTS (Monte Carlo Tree Search), 是一类**概率统计型**树搜索算法的统称, 可以较为有效地解决一些探索空间巨大的问题, 相比于传统的深度优先或者广度优先算法, MCTS 执行一种非对称的树的适应搜索空间拓扑结构的生长, 这个算法会更频繁地访问更有趣的节点, 并聚焦其搜索时间在更加相关的树的部分, 算法流程一般分为四步: 选择、拓展、评估、回传



落子选择

落子评估

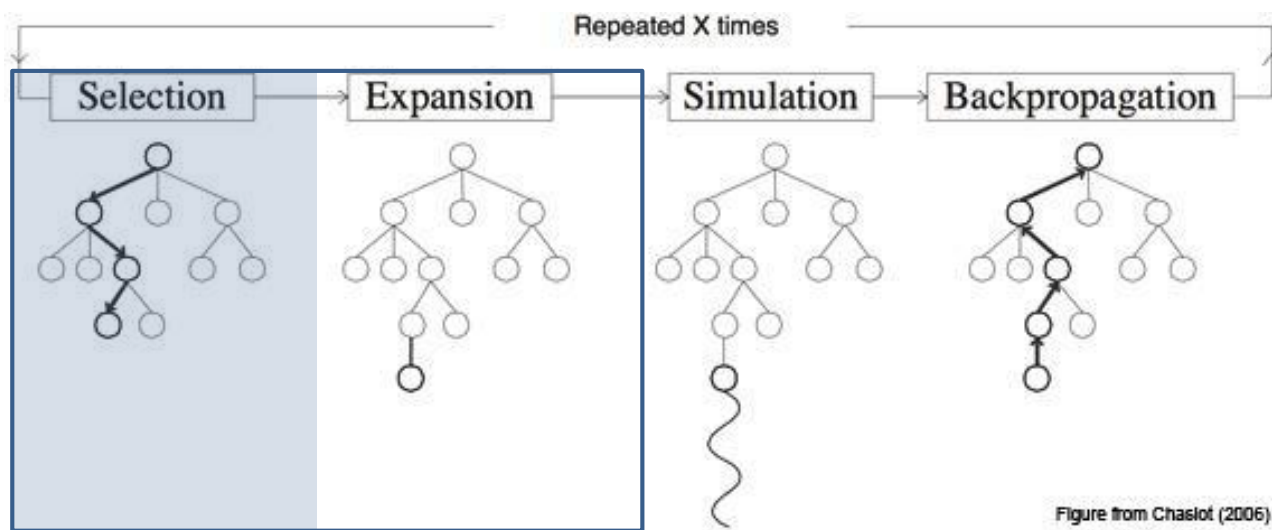
◆ 算法流程

总共对当前状态 S_t 进行K次采样, 在每次采样的过程中:

- 评估: 从上面这个新的子节点开始, 用一个简单策略比如随机策略走到底, 得到一个胜负结果。这种简单策略一般是决策很快但可能不是很精确的策略, 因为如果这个策略走得慢, 结果虽然会更准确, 但由于耗时多了, 在单位时间内的模拟次数就少了, 所以不一定会棋力更强, 有可能会更弱。

围棋AI求解框架：MCTS

MCTS (Monte Carlo Tree Search), 是一类**概率统计型**树搜索算法的统称, 可以较为有效地解决一些探索空间巨大的问题, 相比于传统的深度优先或者广度优先算法, MCTS 执行一种非对称的树的适应搜索空间拓扑结构的生长, 这个算法会更频繁地访问更有趣的节点, 并聚焦其搜索时间在更加相关的树的部分, 算法流程一般分为四步: 选择、拓展、评估、回传



落子选择

落子评估

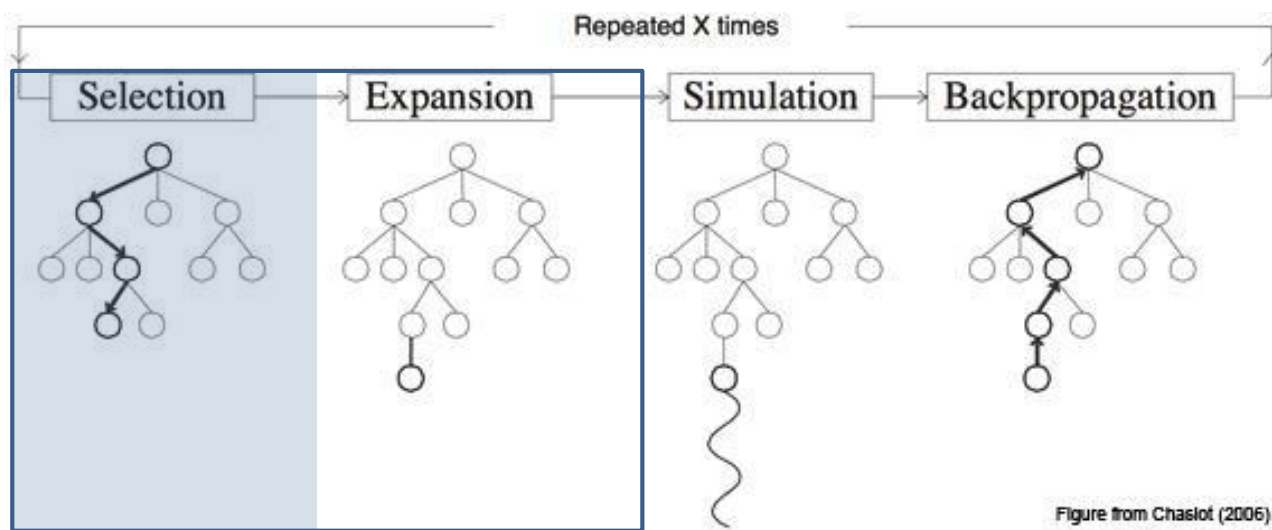
◆ 算法流程

总共对当前状态 S_t 进行K次采样, 在每次采样的过程中:

- 回传: 在评估的过程中, 我们通过简单策略得到了当前新加入的子节点的一个价值估计, 如围棋中就是最终的胜负结果, 我们将在本步中用得到的价值估计更新本次采样中到达该新子节点的路径上的所有节点, 一般将路径上节点的访问次数加1, 同时节点的价值根据回传的价值做更新, 在围棋中节点价值的更新一般指节点的胜利次数根据模拟的最终结果加1 (胜利) 或-1 (失败)

围棋AI求解框架：MCTS

MCTS (Monte Carlo Tree Search)，是一类**概率统计型**树搜索算法的统称，可以较为有效地解决一些探索空间巨大的问题，相比于传统的深度优先或者广度优先算法，MCTS 执行一种非对称的树的适应搜索空间拓扑结构的生长，这个算法会更频繁地访问更有趣的节点，并聚焦其搜索时间在更加相关的树的部分，算法流程一般分为四步：选择、拓展、评估、回传



落子选择

落子评估

◆ 算法流程

- 总共对当前状态 S_t 进行K次采样，在K次采样结束后，我们会得到当前根节点下不同动作对应的子节点的信息，如各个子节点的价值和访问次数，最终我们会根据这些子节点的信息来选择真正输出的动作，如选择访问次数最多的子节点对应的动作作为输出
- 在围棋中，整个MCTS的过程就相当于在看到当前的棋局时前向推演了K次，再根据K次的推演最终决定当前局面自己的落子位置

围棋AI求解框架：MCTS

- ◆ MCTS各种变体算法的主要区别之一是采样中的选择部分的策略，在围棋中，一般采用UCB (Upper Confidence Bound) 算法来进行节点的选择，对应的MCTS变体方法被称为UCT (Upper Confidence Bound Applied to Trees) 算法
- ◆ 每个节点会保存两类信息，节点的价值和节点的访问次数。节点的选择过程会综合这两类信息来考虑，有的节点的胜利次数少但未必就不是一个好的选择，这可能是因为该节点的访问次数少导致的，因此节点的选择过程应该是一个平衡探索（访问次数）和利用（胜利次数）的过程，经典的就是UCB算法

$$a_t = \operatorname{argmax}_a \left(\frac{w_i}{n_i} + c \frac{\sqrt{\log N_i}}{n_i} \right)$$

其中， w_i 为节点的胜利次数， n_i 为节点的访问次数， N_i 为当前总的访问次数， c 是一个平衡探索和利用的参数，可以根据经验调整， c 越大则算法越偏向于选择访问次数少的节点，反之则偏向选择胜率大的节点

围棋AI求解框架：MCTS

- ◆ 除了经典的UCT算法，在围棋结合MCTS方法的过程中，往往还引入了许多的专家知识来辅助算法的选择和评估部分，即人为启发式提供落子选择的先验，而不是只考虑节点的胜利次数和访问次数，和人为启发式地推演棋局、评估局面的价值而不是采用随机模拟
- ◆ 例如在选择和评估的过程中，会首先定义一批根据专家知识确立的特征，如考虑气（liberty）、形（pattern）、定式（joseki）、攻击模式（attack patterns）、防守模式（defense patterns）等，根据特征对于不同节点对应的局面进行打分，得到的分值分别辅助落子选择过程和评估中的模拟走子过程，从而提升MCTS算法的性能

专家知识：手动设计特征
AlphaGO：数据驱动学习特征

AlphaGo

落子选择：在本次采样的过程中，选择哪一条分支进行分析评估

落子评估：对于本次采样过程中展开的新节点棋面，输赢的可能性是多少

随着采样的进行，算法构建的博弈树在逐渐展开，节点的价值估计也变得更加准确

	传统MCTS	AlphaGo:深度学习增强版MCTS
落子选择	结合人为规则和节点访问次数进行选择	以 策略网络 评估值进行选择
落子评估	随机推演/考虑气等人为规则定式推演	快速走子网络+估值网络 相结合

策略网络：输入当前的棋局，输出不同候选位置的选择概率，作为先验

快速走子网络：同策略网络，输入当前棋局，输出不同候选位置的选择概率，相比于策略网络，网络的层数更少，以实现快速推理和相对准确的估计之间的平衡

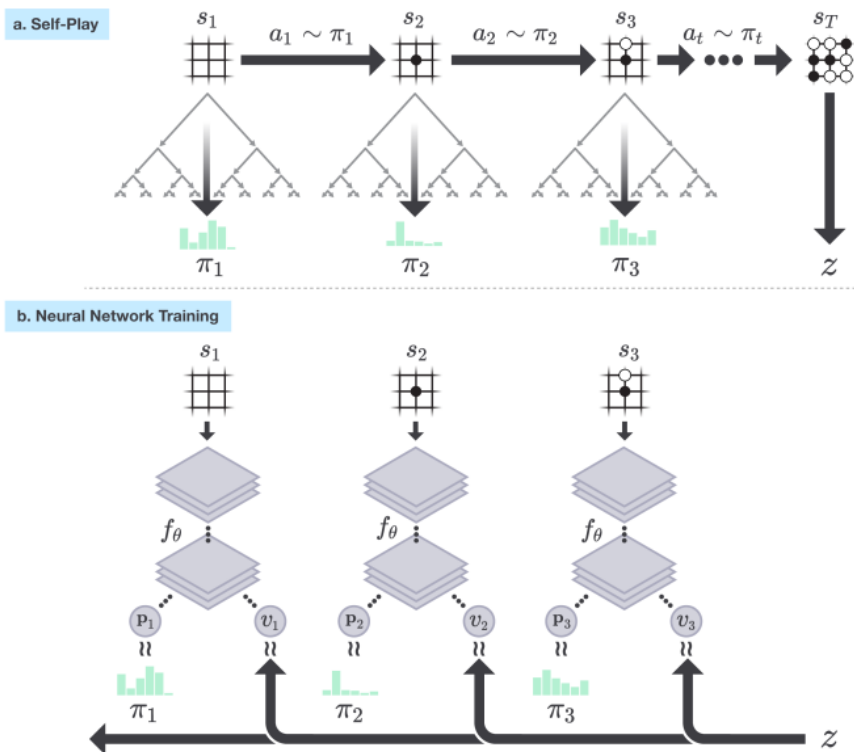
估值网络：输入当前棋局，输出该棋局对应的胜负结果的价值估计

以上策略均可以直接或间接通过人类高质量数据习得

AlphaGo Zero

◆ 和AlphaGo相比:

1. 仍然是MCTS的框架，但是网络的训练不再使用人类数据
2. 采用ResNet自动学习棋盘特征，舍弃手动设计特征。残差网络相比于传统卷积神经网络对于捕捉棋盘这种高度结构化目标信息更为有效，可以得到更好的策略与估值网络（策略与估值网络合并共享底层结构）；



MCTS结合网络的过程:

1. 选择

$$a_t = \underset{a}{\operatorname{argmax}} (Q(s_t, a) + U(s_t, a))$$

$$U(s, a) = c_{\text{puct}} P(s, a) \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)}$$

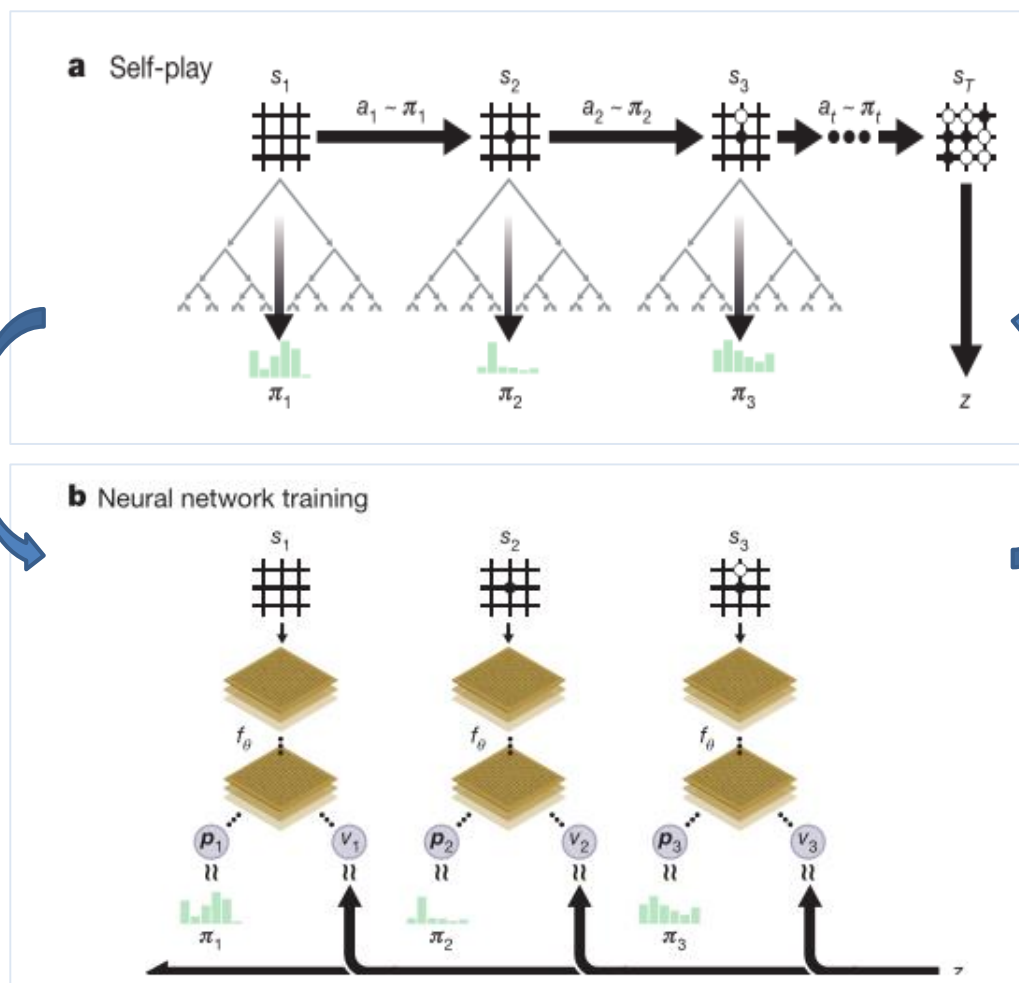
2. 拓展: $N, W, Q = 0, P = P_r$, P 使用策略网络得到
3. 评估: 使用价值网络计算得到节点价值 v
4. 回传: $N = N + 1, W = W + V, Q = W/N$

AlphaGo Zero

网络的训练

Self-play: 初始化策略网络和价值网络，采用自我博弈，双方都使用结合策略网络和价值网络的MCTS方法来进行动作选择，博弈中会记录每局每手MCTS模拟后的落子概率分布和每局的结果（从而已知每手的胜负）

Optimization: 优化时，让每一个状态的落子概率分布，即策略网络的输出和MCTS模拟结果尽可能相近；胜负估计，即价值网络的输出和实际结果尽可能相近



AlphaGo Zero

网络的训练

Optimization: 根据最近产生的500000局selfplay的数据进行策略网络和价值网络的优化；每1000个训练步保存一个网络参数版本

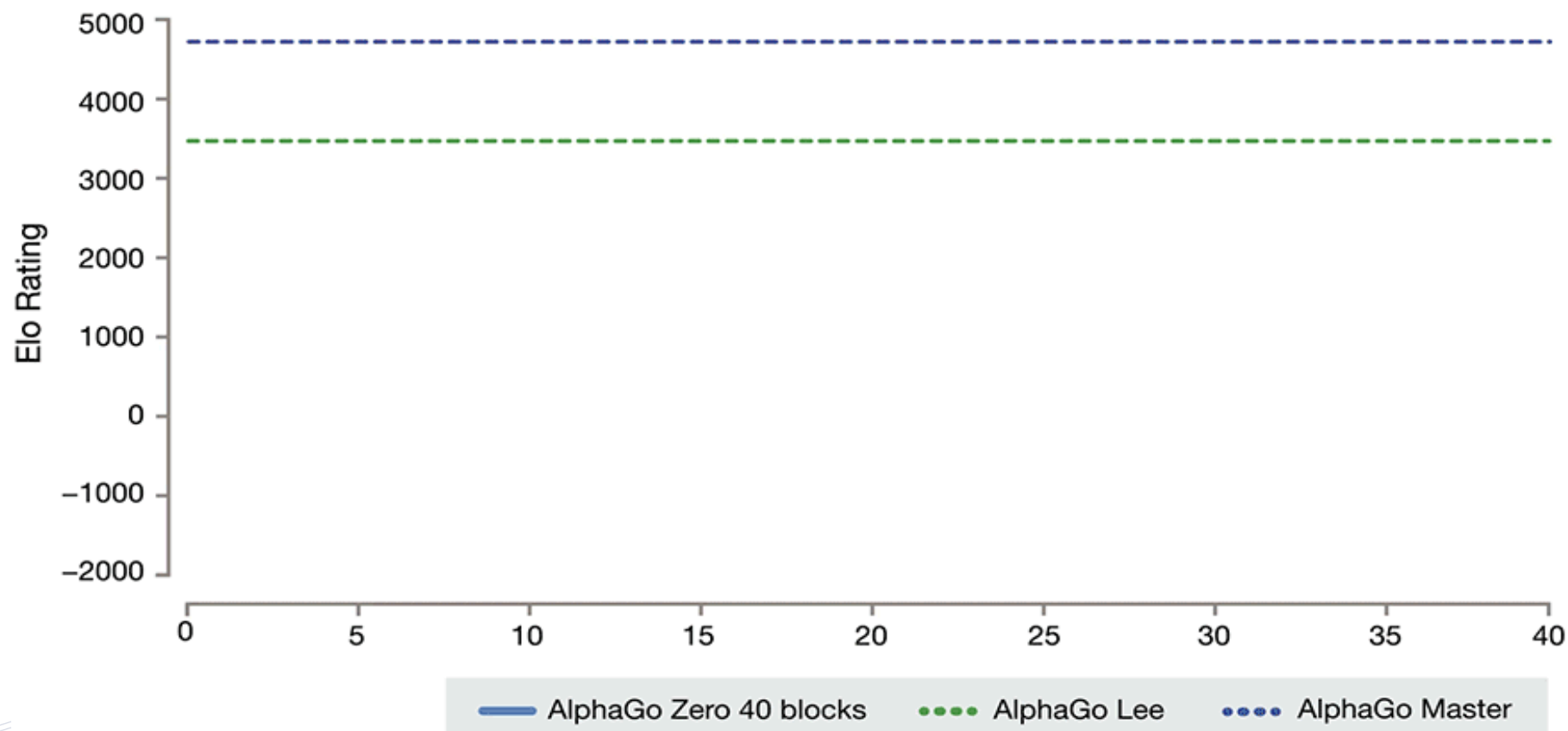


Evaluator: 评价新产生的网络，将其与当前最好的网络对局400次，若胜率超过0.55，则就将其变为当前最好网络



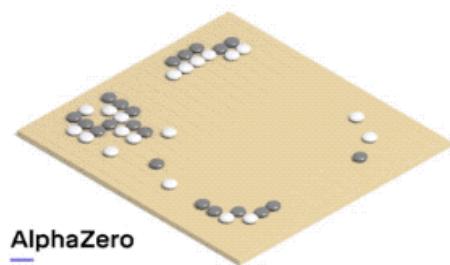
Self-play: 始终使用当前最好的网络进行self-play产生数据

AlphaGo Zero



* AlphaGo Lee 与AlphaGo Master为AlphaGo初代版本之后的两个版本。其中AlphaGo Lee采用更复杂的估值与策略网络，且算力提升；AlphaGo Master采用AlphaGo Zero架构(ResNet模型)，采用人类对抗数据初始化，仍使用定义好的抽象的特征，且落子评估仍然保留rollout过程。

AlphaZero



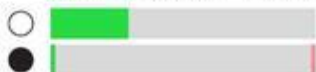
1. AlphaZero为AlphaGo Zero的更通用版本，修改了部分训练细节；这一套框架可以同时学习围棋、象棋和日本将棋
2. 网络需要的训练量：国际象棋需要9小时，将棋需要12小时，围棋需要13天。

Chess



AlphaZero vs. Stockfish

W:29.0% D:70.6% L:0.4%



W:2.0% D:97.2% L:0.8%

Shogi



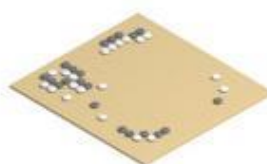
AlphaZero vs. Elmo

W:84.2% D:2.2% L:13.6%



W:98.2% D:0.0% L:1.8%

Go



AlphaZero vs. AGO

W:86.9% L:13.1%



W:53.7% L:46.3%

Stockfish: 第九季国际象棋世界冠军；
Elmo: CSA世界冠军版本；
AGO: AlphaGo Zero。

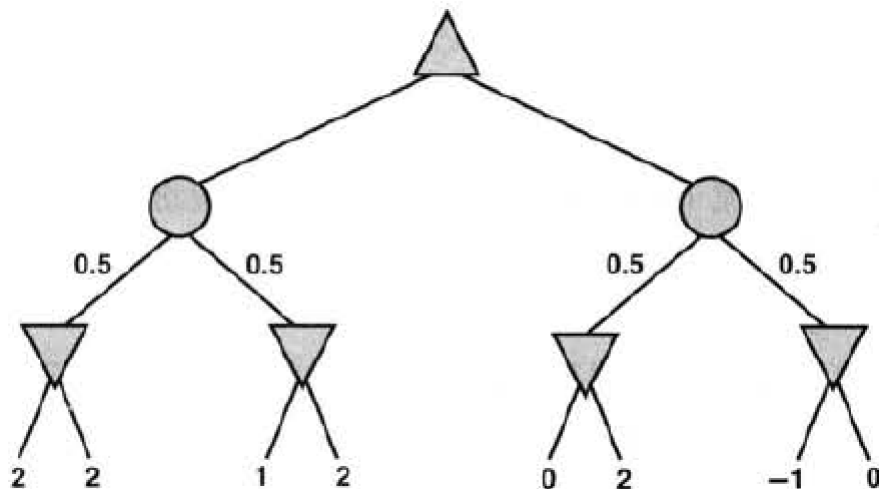
本章小结

- 博弈游戏通过下列元素定义
 - 初始状态，每个状态下的合法行动，每个行动的结果，终止测试，和终止状态上的效用函数
- 在有完整信息的两人零和游戏中，极小极大算法可以通过对博弈树的深度优先枚举选出最优招数。
- α - β 搜索算法可以计算出和极小极大算法一样的最优招数，由于消除了被证明无关的子树，效率得到提高。
- 通常考虑整棵博弈树是不可行的，所以在某个点截断搜索，并应用启发式评估函数对某个状态的效用值进行计算。
- 对于像桥牌和军棋这样的不完整信息的游戏，需要对每个游戏者当前和未来的信念状态进行推理。

课后作业1

- 本题以井字棋（圈与十字游戏）为例练习博弈中的基本概念。定义 X_n 为恰好有 n 个 X 而没有 O 的行、列或者对角线的数目。同样 O_n 为正好有 n 个 O 的行、列或者对角线的数目。效用函数给 $X_3 = 1$ 的棋局 +1, 给 $O_3 = 1$ 的棋局 -1。所有其他终止状态效用值为 0。对于非终止状态, 使用线性的评估函数定义为 $Eval(s) = 3X_2(s) + X_1(s) - (3O_2(s) + O_1(s))$ 。
 - a. 估算可能的井字棋局数。
 - b. 考虑对称性, 给出从空棋盘开始的深度为 2 的完整博弈树 (即, 在棋盘上一个 X 一个 O 的棋局)
 - c. 标出深度为 2 的棋局的评估函数值。
 - d. 使用极小极大算法标出深度为 1 和 0 的棋局的倒推值, 并根据这些值选出最佳的起始行棋。
 - e. 假设结点按对 $\alpha - \beta$ 剪枝的最优顺序生成, 圈出使用 $\alpha - \beta$ 剪枝将被剪掉的深度为 2 的结点。

课后作业2



上图给出了一个简单游戏的完整博弈树。假设叶结点的计算顺序是从左到右，而且在一个叶结点被评估之前我们对它一无所知，可能的取值范围是 $-\infty$ 到 ∞

课后作业2

- a. 复制这个图，在图中标出所有内部结点的值，用箭头指出根结点选择的行棋。
- b. 给定前六个叶结点的值，还需要计算第七个和第八个叶结点的值吗？如果是给定前七个叶结点的值，第八个还需要计算吗？请对你的结论给出解释。
- c. 假设叶结点的值都在-2到2之间。计算完前两个叶结点值之后，左手机会结点的取值范围是多少？
- d. 用圆圈划出在c中假设下无需计算的叶结点。

THANKS